

ICTPRG432 Develop data-driven applications

ICTPRG433 Test software developments

ICTPRG430 Apply introductory object-oriented language skills

Learner version

Assessment task 2

Develop data-driven apps

Disclaimer: This resource was developed by Holmesglen Institute of TAFE in 2024 for the State of Victoria (Department of Education and Training) for educational purposes, is © State of Victoria (Department of Education and Training) and is available under a Creative Commons Attribution Non-commercial 4.0 International ([CC BY-NC 4.0](https://creativecommons.org/licenses/by-nc/4.0/)) license.

The license does not apply to: Trademarks or branding, including the Victorian Government logo and any unattributed content supplied by third parties

Learner version

Assessment task 2

Develop data-driven apps

Learner information

Learner name: Finnley Newnham

Learner ID: 100721115

Section A – Program/course details

Qualification code: ICT40120	Qualification title: Certificate IV in Information Technology (Programming)
Unit code:	Unit title:
CPRO2 Data Driven Programming	
ICTPRG432	Develop data-driven applications
ICTPRG433	Test software developments
ICTPRG430	Apply introductory object-oriented language skills
Pre/co-requisites: N/A	

Section B – Assessment task details

Assessment number: 2 of 2	Semester/year: 2/2026	Due date: Session 17
Duration of assessment: 14 Sessions	Assessment task results: This assessment task will be marked as: ☒ Ungraded result: <i>satisfactory</i> or <i>not yet satisfactory</i>	

Section C – Instructions to learners

Assessment task description:

This assessment is comprised of 3 parts.

Part 1: Knowledge questions
Part 2: Develop a 2D-Shapes Calculator
Part 3: Develop “ToDo” List

Part 2 also covers software testing, including both manual and automated testing. It provides instructions to:

- Identify unit test requirements (including the test framework)
- Identify system test requirements
- Explore test-case design techniques
- Develop a test plan
- Validate the test results with a member of the software development team
- Learners are required to answer all questions correctly and to a satisfactory level. If this is not achieved on the first attempt, a **resubmission** will be required.

Section C – Instructions to learners

- You must demonstrate the application (Part 2, Part3) on the assessment due date. You may choose to present it either in a breakout session or during the class.
- Once learners have completed all the questions, the assessment must be uploaded and submitted along with the signed assessment coversheet via Brightspace.
- Please **check your spelling and grammar** before submitting the final version to the assessor.
- If you require **special consideration or adjustments**, discuss your needs with your assessor.
- You may consult:
 - Class learning materials on **Brightspace**
 - Other soft copy resources
 - Reliable online sources
- However, all responses must be written in your own words. If you include direct quotes, you must cite the original source.
- Your completed assessment must be uploaded to Brightspace along with a completed and signed coversheet by the due date.
- Submit all required working files, documentation, and supporting materials in a zipped folder, including the signed coversheet.
- If you are unable to submit by the due date, you must obtain written approval from your assessor for alternative submission arrangements.
- Learners must adhere to organisational policies and standards, including those related to:
 - Intellectual property
 - Privacy laws
 - Plagiarism and academic honesty

Further details are available at <https://holmesglen.edu.au/Students/Student-Resources>

Section D – Conditions for assessment

Unit application:

- This is an individual task.
- You must meet all criteria listed in the marking guide to be satisfactory in this task.
- You must submit all required working files, documentation, and any other assets that you feel may be required in a zipped file, including the completed and signed coversheet. The assessment must be completed and submitted electronically to Brightspace by the due date. If this is not possible, you must contact your assessor to gain written approval for an alternative arrangement for submitting the assessment.
- If not successful within the enrolment period as per Holmesglen assessment procedure, you will be requested to resubmit within 7 days of receiving feedback. You will have the opportunity to resubmit if any part of the assessment is deemed unsatisfactory (you are permitted **TWO (2)** resubmission per assessment task). Resubmissions must be submitted by the resubmission due date provided by your teacher.
- This task is open book. You may use the internet for research purposes only. All answers must be in your own words. Where a quote is used, you must cite the information source.
- If you feel you require special allowance or adjustment to this task, please discuss with your assessor within one week of commencing this assessment. Any change to assessment arrangements must be reviewed by the Education Manager and approved by the Head of Department.
- You can appeal an assessment decision according to the Holmesglen Assessment Complaints and Appeals Procedure.
- You are expected to dedicate time to develop this assessment task both in and out of the classroom.
- Learner must contribute to and abide by organisational standards including intellectual property, privacy laws, and plagiarism and academic honesty. Further information is detailed at: <https://holmesglen.edu.au/Students/Student-Resources>
- **Use of Generative AI Tools in Assessments**
You are permitted to use generative AI tools to support your research and development process for this assessment. This aligns with the broader educational focus on digital literacy and emerging technologies. However, your use of these tools must be **transparent, responsible, and in accordance with academic integrity standards**.

You must not use generative AI to:

1. Generate full-text responses for written assessment questions.
2. Write complete scripts or presentations without your own input.
3. Produce entire data sets, budgets, or analytical outputs without demonstrating your own understanding.

Section D – Conditions for assessment

Your use of these tools should be to support your work, not to replace it. This includes tasks such as:

1. Brainstorming ideas.
2. Conducting preliminary research.
3. Structuring your presentation.
4. Editing and refining your written work.

Academic Honesty and Plagiarism

Any use of AI tools must be clearly acknowledged. For example, you can include a note such as:

“Generative AI was used to assist with brainstorming and editing.”

Failure to follow these guidelines may be considered academic misconduct under Holmesglen’s policy.

Equipment/resources learners must supply:	Equipment/resources to be provided by the RTO:
Learners opting to BYOD laptop or intending to learn remotely will require access to: A MAC or PC/laptop with the following minimum specification: • Quad-Core CPU, 16GB of RAM, 250GB of Storage, 2 GHz or faster processor • Windows 10 OS, or virtual machine installed with Windows 10 OS (available free from https://developer.microsoft.com/en-us/windows/downloads/virtual-machines/ or https://www.microsoft.com/en-us/evalcenter/evaluate-windows-10-enterprise) • Headset with microphone and webcam • Internet access Applications access available through Holmesglen MyHorizon and free access which can be downloaded via link below: • Brightspace (Learning Management System) - https://holmesglen.brightspace.com/ • 365 Microsoft office suite - https://portal.office.com • WebEx - https://holmesglen.webex.com/ • Holmesglen OneDrive • Google Chrome – recommended web browser • 7-Zip - https://www.7-zip.org/download.html • Microsoft Office • Lucid chart via internet • Microsoft Visual studio Enterprise/community edition 2017/19/21/22 or latest version available in the market with C# components installed as a language • SQLite DB Browser – free to download USB drive/External hard drive / cloud storage to save project work	This is a blended learning course and as such a remote learner will access their own computer equipment as per the specification provided. The Institute will provide the following: Holmesglen computer classrooms: • data projector, whiteboard, computer with double screens, mobile Cisco Spark Board, Conference camera HP Z1 Entry Tower G5 • Dual Displays 24" HP Monitor • CPU: Intel Core i7-9700K @ 4GHz • Motherboard: 8591 • RAM 32GB • 1TB SSD • OS: Windows 10 iMAC • iOS Catalina 10.15.7 (19H1713) • 27" Retina 5K • Processor: 4GHz Quad-Core Intel Core i7 • Memory: 16GB 1867 MHz DDR3 This will vary as CAIT upgrades computer rooms. • Internet connection Applications access available at ZENworks and Holmesglen MyHorizon: • Brightspace (Learning Management System) • 365 Microsoft office suite • Lucid chart • WebEx • Google Chrome – recommended web browser • 7Zip • Applications: • Microsoft Office • WebEx - free to download • Microsoft Visual studio Enterprise/community edition 2017/19/21/22 or latest version available

Equipment/resources learners must supply:	Equipment/resources to be provided by the RTO:
	in the market with C# components installed as a language • SQLite DB Browser * All are licensed

Section E – Assessment questions/criteria

Assessment number: 2

Assessment title: Develop data-driven apps

Learner name: Finnley Newnham

Learner ID: 100721115

Unit code:

Unit title:

CPRO2 Data Driven Programming

ICTPRG432

Develop data-driven applications

ICTPRG433

Test software developments

ICTPRG430

Apply introductory object-oriented language skills

Date: 16/07/2026

Part 1 – Knowledge Questions (Written Response)



AI Assisted Learning Permitted

Guided Use

You may use AI tools to help research, clarify concepts, or check your understanding. However, your final response must demonstrate your own application and comprehension.

Provide your responses in the boxes below each question.

Question	Satisfactory response
What is a class and constructor in Object Oriented Programming (OOP) • Discuss the advantages of classes • How do classes benefit software development	Yes ☐ No ☐
Answer: A class is like a blueprint or template used to create objects. It defines what an object “knows” (<i>fields</i>) and “can do” (<i>methods</i>). For example, a Cat class may have fields such as “breed” or “owner”, and have methods such as “eat” or “sleep”. A constructor is a special method inside a class that is automatically called when a new object is created. Its purpose is to initialise the object's data with default values or values provided. Constructors help ensure that every object starts in a valid state. There are many advantages to using classes. Firstly, they organise related data and functions into single units and make programs easier to read, maintain, and update. They promote code reuse by allowing the same class to create many objects, and allow for polymorphism and inheritance to further reduce code duplication and increase	Comment:

ICTPRG432 Develop data-driven applications

ICTPRG433 Test software developments

ICTPRG430 Apply introductory object-oriented language skills

Assessment task 2: Develop data-driven apps

Question	Satisfactory response
modularity. They also support encapsulation, which helps protect data from being accessed or modified incorrectly, making code easier to maintain and debug. Classes make software development more efficient by encouraging modular and reusable code. Developers can build applications using well-organised components instead of repeating the same code multiple times. This reduces errors, simplifies testing and debugging, and makes it easier to add new features or modify existing ones without affecting the rest of the program. Thus, classes improve the maintainability, reliability, and scalability of software.	

Question	Satisfactory response
What is Object Aggregation (the “is-a” or “part-of”). You may use an example to explain.	Yes ☐ No ☐
Answer: Object Aggregation is a “has-a” relationship between objects, where one class “has” another object within it. Notably, if the parent object is destroyed, the child object persists. An example may be a Car class that has a Wheel object. If the instance of the Car is destroyed, the Wheel object will still remain. This type of relationship is loosely coupled.	Comment:

Question	Satisfactory response
Discuss the 3 basic programming structures, sequential, condition, loop which can be implemented through Object oriented data structure called class.	Yes ☐ No ☐
Answer: A sequential structure executes statements one after another in the order they are written, and every program uses sequential execution as its basic flow. A “condition” structure allows the program to make decisions based on whether a condition is true or false, which is typically achieved using <i>if</i>, <i>else if</i>, <i>else</i>, or <i>switch</i> statements. A “loop” structure repeats a block of code until a condition is met, and common loop statements include <i>for</i>, <i>while</i>, and <i>foreach</i>. These three structures can be implemented in classes’ methods. A method is a block of code that executes some logic, and may return data. They will use sequential programming by executing logic in order (<i>ie. cleaning data, then adding it to a database</i>). They may use conditions to change behaviour or the outcome of the action (<i>ie. check connection to the database - if failed, return an error message, else continue with action</i>). They may use a loop to achieve the method’s expected behaviour without code duplication (<i>ie. to add data to a database, loop through provided data until all is added</i>). By utilising these structures, classes can implement efficient methods that are functional, easy to read, and scalable.	Comment:

Question	Satisfactory response
What is member field and member method in a C# class Note member field can be a primitive instance variable and member method is also refer to as instance method in some other languages	Yes ☐ No ☐
Answer: A member field is a variable declared inside a class that stores information about an object; fields are what the object “knows”. For example a Cat class may have a “hunger” field, and an instance of that class may have hunger = “5”. A member field can be a primitive data type such as int, double, bool, or string, or it can be a reference to another object. Each object created from the class has its own copy of its instance member fields. A member method is a function defined inside a class that performs actions using the object's data; methods are what the object “can do”. The method may take in a parameter containing additional provided data, and may or may not return data. Member methods can read or modify the values stored in the member fields and define the behaviour of the object. For example, the Cat class may have an “eat” method that, when called, adds 2 to the hunger property and prints the current hunger value to the terminal.	Comment:

Question	Satisfactory response
What is static member field and static member method in a C# class Note static member field is also referred as class variable and static member method is also refer to as class method in some other languages	Yes ☐ No ☐
Answer: A static member field is a variable that belongs to the class itself, rather than to individual objects created from the class. Only one copy of a static field exists, and it is shared by all objects of that class. Static member fields are commonly used to store information that should be the same for every object, such as a counter or a constant value. For example, the Cat class may have a static field for NumberOfCats that keeps track of how many Cat objects there are. A static member method is a method that also belongs to the class rather than an individual object. It can be called without creating an instance of the class by using the class name. Because it is not associated with a specific object, a static method can only directly access other static members of the class. The Cat class may have a static method for PrintCatCount that prints NumberOfCats to the terminal. By making this method and field static, no actual instance of a Cat object needs to exist to retrieve this information.	Comment:

Question	Satisfactory response
Discuss the debugging and testing approaches and techniques in software development	Yes ☐ No ☐
Answer: Debugging is the process of finding, identifying, and fixing errors in a program, and there are many common debugging techniques. Breakpoints can be used to pause the program, inspect variables, and step through code one line at a time to help pinpoint the source of a bug. Reading error messages and exception details can also help to identify	Comment:

Question	Satisfactory response
the cause of a problem, or adding temporary output statements (<i>ie. with Console.WriteLine() in C#</i>) to display variable values at runtime can also help. Testing is the process of checking that software works correctly and meets its requirements. Testing helps identify defects <i>before</i> the software is released and ensures that new changes do not introduce additional problems. Common testing approaches will usually include Unit testing, which are specialised functions that automatically test individual methods or classes to ensure they work correctly. Further manual testing through quality assurance may also be used, depending on the scale of the program/business.	

Question	Satisfactory response
What is the Software Development Life Cycle (SDLC), and why is it important in software development? Describe the key phases involved and explain how each phase contributes to the successful creation of software.	Yes ☐ No ☐
Answer: The Software Development Life Cycle is a structured process used to plan, design, develop, test, deploy, and maintain software. It has a step-by-step framework that helps development teams produce high-quality software that meets user requirements and reduces risks, costs, and errors. The SDLC is important because it ensures that software is developed in an organised and systematic way. This standardised approach improves communication between team members, helps identify problems early, and makes projects easier to manage and maintain. The key phases of the SDLC are: 1. Planning The planning phase identifies the project's goals, scope, budget, timeline, and required resources. This phase ensures the project is feasible, establishes a clear development plan, and helps reduce the possibility of scope creep. 2. Requirements Analysis During this phase, developers gather and document the needs of the users and stakeholders. These requirements define what the software must do and provide a foundation for the rest of the project. 3. Design The design phase creates the software's structure, including the user interface, database, system architecture, and program components. Good design makes the software easier to build, test, and maintain. 4. Development/Implementation Developers write the program code based on the design specifications. Features are implemented, databases are created, and the application is built. 5. Testing The software is tested to identify and fix defects. Different types of testing (<i>ie. unit testing</i>) ensure that the software functions correctly and meets the requirements. 6. Deployment Once testing is complete, the software is released to users. Deployment may involve installing the application, configuring databases, and making the software available for use. 7. Maintenance After deployment, the software is monitored and updated to fix bugs, improve	Comment:

Question	Satisfactory response
performance, and add new features as user needs change. Maintenance helps keep the software reliable and secure over time.	

Question	Satisfactory response
List and describe common types of documentation used in the Software Development Life Cycle (SDLC). For each type, explain its purpose and how it supports the development process	Yes ☐ No ☐
Answer: Documentation is an important part of the Software Development Life Cycle (SDLC). It records information about the software throughout each stage of development and helps developers, testers, clients, and future maintainers understand the system. 1. Requirements Documentation Requirements documentation describes what the software must do, including functional and non-functional requirements. It ensures that developers and stakeholders have a shared understanding of the project's goals and expected features, and allows these requirements to be referred to throughout development. 2. Design Documentation Design documentation explains how the software will be built. It may include system architecture diagrams, database designs, class diagrams, and user interface designs. This documentation provides developers with a clear blueprint for implementation and may help with task delegation. 3. Technical Documentation Technical documentation describes how the software works internally. It includes source code comments, algorithms, database schemas, API documentation, and configuration details. It helps developers understand, maintain, and update the software. 4. Test Documentation Test documentation records the testing process, including test plans, test cases, test results, and bug reports. It ensures that all features have been tested and helps verify that the software meets its requirements before release. 5. User Documentation User documentation explains how to use the software. It may include user manuals, installation guides, quick-start guides, or online help. Its purpose is to help end users operate the application correctly and efficiently. 6. Maintenance Documentation Maintenance documentation records software updates, bug fixes, known issues, and version changes after the software has been deployed. It supports future maintenance by making it easier for developers to troubleshoot problems and implement improvements.	Comment:

Question	Satisfactory response
Discuss the typical development process for small-size software applications. What steps are usually involved, and how might they differ from those used in large-scale projects?	Yes ☐ No ☐
Answer: The development process for a small software application would usually follow the same basic principles as the Software Development Life Cycle (SDLC), but may be simpler, faster, and involve fewer people. Small projects may have a single developer or a small team, meaning decisions can be made quickly and communication becomes easier. Because the team is small, communication may be informal, and notably, documentation may be significantly less. In contrast, large-scale projects involve many developers and stakeholders, require detailed planning and documentation, follow more formal project management processes, and include extensive testing and quality assurance. Smaller companies or projects would not have these resources and may not need them due to the scale of their work.	Comment:

Question	Satisfactory response
Discuss the testing processes, tools, and techniques commonly used for small-size software applications. How do these ensure the application works correctly and meets user needs?	Yes ☐ No ☐
Answer: For small software applications, testing is often performed by the developer using a combination of manual and automated techniques, very similar to larger-scale projects. Unit testing checks individual methods or functions to ensure they perform as expected, and depending on the software, these are still likely to be used. System testing evaluates the complete application as a whole, and in a small team, the developer is likely to conduct these tests themselves (<i>as opposed to QA personnel</i>). Boundary value testing - which checks how the application handles values at the limits of valid input - and Exception testing - which verifies that the application correctly handles errors without crashing - are also likely done by the development team. These testing processes, tools, and techniques help ensure that the application is reliable, functions as intended, and meets user needs by identifying any issues before delivering the software to the users. These processes are very similar to what a large-scale company may implement, but at a reduced scale and intensity.	Comment:

Question	Satisfactory response
Research industry standard auto test frameworks for the following languages	Yes ☐ No ☐
Answer: As no languages were listed, I investigated some industry-standard auto test frameworks for C# and Python.	Comment:

Question	Satisfactory response
C# 1. <i>MSTest</i> MSTest is Microsoft's official testing framework for .NET applications. It is included with Visual Studio and integrates well with the .NET development environment. MSTest supports unit testing, assertions, and automated test execution, making it a popular choice for C# projects. 2. <i>NUnit</i> NUnit is one of the oldest and most widely used open-source testing frameworks for C#. It provides a rich set of assertions, supports parameterised tests, and works with many .NET applications. NUnit is commonly used in both industry and education because it is flexible and easy to use. 3. <i>xUnit.net</i> xUnit.net is a modern, open-source testing framework designed specifically for .NET applications. It is widely used in professional software development and is the preferred testing framework for many new .NET projects. xUnit supports automated unit testing and integrates with Visual Studio and continuous integration (CI) tools. Python 1. <i>pytest</i> pytest is the most popular testing framework for Python. It is open-source, easy to use, and supports simple unit tests as well as complex test suites. pytest provides powerful features such as fixtures, parameterised testing, and detailed test reports, making it widely used in professional software development. 2. <i>unittest</i> unittest is Python's built-in testing framework and is included with the Python Standard Library. It supports creating and organising unit tests, test fixtures, assertions, and automated test execution. Because it requires no additional installation, it is often used for both learning and professional projects.	

Question	Satisfactory response
Describe the software testing concept of IPO (Inputs, Process, Outputs). How can this model be used to design and test applications effectively?	Yes ☐ No ☐
Answer: The IPO (<i>Inputs, Process, Outputs</i>) model is a simple method used to understand, design, and test software applications. It breaks a program into three main stages, where the data entered into the program (<i>inputs</i>), the operations are performed on that data (<i>process</i>), and the results are produced by the program (<i>outputs</i>).	Comment:

Question	Satisfactory response
Inputs are the data or information provided to the application. These can come from user input, files, databases, or other systems. Then, the process is the logic the application performs on the input data. Some examples may include validating the input, performing calculations, storing data in a database, or updating records. Finally, outputs are the results produced by the application after processing the input. Outputs can be messages displayed to the user, reports, updated database records, or exported files. The IPO model can be used to design and test applications by examining each stage in isolation. Test cases are created by supplying different inputs, checking that the application processes the data correctly, and verifying that the expected outputs are produced.	

Question	Satisfactory response
Explain the following data structure - array, tree	Yes ☐ No ☐
Answer: An array is a data structure that stores a collection of elements in a fixed-size sequence with a specific position at an index. Arrays are useful when storing a list of related data - such as numbers, names, or objects - and because the elements can be accessed using an index, arrays are very efficient. A tree is a hierarchical data structure made up of nodes connected by branches. The top node is called the root, each node can have one or more child nodes, and a node with no children is called a leaf. Trees are commonly used to represent hierarchical information, such as file systems or menu structures, and they also provide efficient searching, sorting, and data organisation in many applications.	Comment:

Question	Satisfactory response
Discuss at least 5 organisational guidelines and procedures applicable to software development and testing frameworks	Yes ☐ No ☐
Answer: Organisational guidelines and procedures are important to development and testing workflows to ensure a project progresses smoothly. Below are some valuable standards and procedures developers may implement. Coding Standards Coding standards define rules for writing source code, including naming conventions, formatting, indentation, comments, and code organisation. Following coding standards improves code readability and makes it easier for multiple developers to understand and maintain software. Version Control Procedures Version control systems, such as Git, are used to track changes to source code. Organisational procedures may require developers to use branches, write meaningful	Comment:

Question	Satisfactory response
commit messages, and review changes before merging them. This helps prevent conflicts, protects previous versions of the code, and supports collaboration. Testing Procedures Testing procedures specify how software should be tested before release. They may require unit testing or the use of automated testing frameworks to ensure that new code does not introduce defects. Documentation Standards Documentation standards ensure that important project information is recorded consistently. This includes requirements documents, design documents, code comments, user manuals, and test reports. Error Handling and Logging These standards dictate how a company will ensure proper exception handling and logging, such as when to suppress an error vs. when to show the user. Developers may standardise the use of techniques such as <i>try-catch</i> blocks or other approaches to handle errors gracefully and log important information. Security Guidelines Security procedures help protect software and user data. These guidelines include validating user input, using parameterised SQL queries to prevent SQL injection, controlling access to sensitive information, and keeping software dependencies up to date.	

Question	Satisfactory response
Discuss software defect management process	Yes ☐ No ☐
Answer: The software defect management process is a structured approach used to identify, record, investigate, fix, verify, and monitor bugs found during software development and testing. The goal is to ensure that defects are resolved efficiently and that the software meets quality standards before it is released. First is the defect identification, where a bug is discovered during testing or by users after the software has been released. Defect reporting is next, where the defect is recorded in a tracking system or bug report. The report should include details such as a description, steps to reproduce, expected vs. actual results, severity and priority of the defect, and screenshots or error messages. Defect analysis will then be done, where developers investigate the bug to determine its root cause. Defect resolution can then be done, where the source code is changed to correct the defect and the fix is tested to ensure it addresses the problem without introducing new issues. Defect Verification further tests the software after a bug fix to confirm that the defect has been fixed successfully and that existing features still work correctly after the change. Finally, defect closure can be done once the bug has been verified as fixed and no further issues are found, where the issue is marked as closed in the defect tracking system.	Comment:

Question	Satisfactory response
Discuss relational database management systems, functions and tools. You may use any database management systems you have used for discussion. For example MS SQL Server, SQLite DB Browser, PHPMyAdmin ...	Yes ☐ No ☐
Answer: A Relational Database Management System is software used to create, store, organise, and manage data in tables. Each table consists of rows and columns, and relationships between tables are created using keys such as primary keys and foreign keys. An RDBMS uses Structured Query Language (SQL) to perform operations such as creating tables, inserting data, updating records, deleting records, and querying information. I have previously used PHPMyAdmin to perform tasks with databases, which is a popular web-based administration tool for MySQL and MariaDB databases. It provides a graphical interface for creating databases, managing tables, executing SQL queries, and importing and exporting data. It is widely used in web development because it runs in a web browser and is more intuitive than using SQL on the command line.	Comment:

Question	Satisfactory response
Discuss the concept of application programming interfaces. What is operation system API, libraries API, web API?	Yes ☐ No ☐
Answer: An Application Programming Interface (API) is a set of rules, functions, and protocols that allows different software applications or components to communicate with each other. With the use of APIs, developers are able to use existing functionality without needing to understand or rewrite the underlying implementation. This speeds up development, improves code reuse, and ensures software components can work together. Operating System API An Operating System API provides functions that allow applications to interact with the operating system. Developers use OS APIs to perform tasks such as reading and writing files, creating windows, accessing hardware devices, managing memory, and handling user input. Library API A Library API is a collection of pre-written classes, methods, or functions that developers can use within their own programs. Instead of writing common functionality from scratch, developers call the library's API to perform specific tasks. Web API A Web API allows applications to communicate with each other over the internet and are commonly used to share data and services between different applications. One application sends a request to the Web API, which processes the request and returns data, often in JSON or XML format.	Comment:

Question	Satisfactory response
Research and discuss at least three different data providers or access technologies that allow programs to connect and interact with databases. For each one, explain the vendor or technology, what type of databases it connects to, and how it is used in programming (e.g., ADO.NET/ Entity Framework, ODBC, PDO).	Yes ☐ No ☐

Question	Satisfactory response
Answer: Data providers and database access technologies allow software applications to communicate with databases. They provide the functionality needed to open database connections, execute SQL queries, retrieve data, and update records. ADO.NET / Entity Framework The vendor for this technology is Microsoft (.NET), and it allows interaction with SQL Server, SQLite, MySQL, PostgreSQL, and Oracle, along with other databases through compatible providers. ADO.NET is Microsoft's data access technology specifically for .NET applications. It allows developers to connect to databases, execute SQL commands, retrieve results, and manage database connections using classes such as Connection, Command, DataReader, and DataAdapter. Entity Framework (EF) is an Object-Relational Mapper (ORM) built on top of ADO.NET. Instead of writing SQL queries directly, developers work with C# objects and classes, while Entity Framework automatically generates the required SQL commands. By abstracting away the SQL, development can become quicker and more intuitive. ODBC (Open Database Connectivity) Once again, this technology's vendor is Microsoft, and similarly interacts with SQL Server, SQLite, MySQL, PostgreSQL, Oracle, and many others through ODBC drivers. ODBC is a standard database access technology that allows applications to communicate with many different database systems using a common interface. Developers install an ODBC driver for the required database, and the application communicates with the database through that driver. ODBC is widely supported by programming languages such as C#, C++, or Python. Its main advantage is that the same application can often connect to different database systems with minimal code changes. PDO (PHP Data Objects) The vendor for this technology is PHP, and it can be used with SQL Server, SQLite, MySQL, PostgreSQL, Oracle, and other databases with PDO drivers. PDO is the standard database access interface used in PHP. It provides a consistent API for connecting to different database systems, executing SQL queries, and retrieving results. PDO supports prepared statements, which help prevent SQL injection attacks by separating SQL commands from user input - useful in situations where a PHP script handles user input. PDO allows PHP applications to switch between supported database systems with very few code changes, making it a good choice for web development.	Comment:

Question	Satisfactory response
What are the core principles of Structured Query Language (SQL)? Explain how these principles guide the way data is stored, managed, and accessed in a relational database.	Yes ☐ No ☐
Answer:	Comment:

Question	Satisfactory response
Structured Query Language (SQL) is the standard language used to create, manage, and manipulate data in a relational database, and provides a consistent way for applications and users to interact with data while maintaining its accuracy and integrity. The core principles of SQL include the following: 1. Data Definition SQL is used to define the structure of a database. Developers can create, modify, and remove database objects (<i>ie. tables, columns, indexes, etc.</i>) using commands like <i>CREATE</i>, <i>ALTER</i>, and <i>DROP</i>. This principle ensures that data is organised into well-defined tables with appropriate data types and relationships. 2. Data Manipulation SQL allows users to add, retrieve, update, and delete data stored in database tables. Common commands include <i>INSERT</i>, <i>SELECT</i>, <i>UPDATE</i>, and <i>DELETE</i>. These commands enable applications to manage information efficiently while keeping the database up to date. 3. Data Integrity SQL supports constraints that help maintain the accuracy and consistency of data. Examples include primary keys, foreign keys, <i>NOT NULL</i>, <i>UNIQUE</i>, and <i>CHECK</i> constraints. These rules prevent invalid or duplicate data from being stored and help maintain relationships between tables and the accuracy of data stored. 4. Data Security SQL databases provide security features that control who can access or modify data. Permissions and user roles can be assigned to restrict access to authorised users only. This helps protect sensitive information from unauthorised access or accidental changes. 5. Transactions SQL allows related database operations to be grouped into transactions, ensuring they are completed successfully as a single unit. If a single error occurs during a transaction where multiple queries are being executed, the changes can be rolled back to preserve the consistency of the database. For example, this is particularly useful when updating or changing a large number of records at once, to prevent a “half-complete” state. These SQL principles ensure that data is stored in an organised and structured way, managed safely, and accessed efficiently. Database tables are designed with relationships and constraints to maintain data integrity, and SQL queries allow users and applications to retrieve and modify information accurately. Security features protect the data, and transactions further ensure that database operations remain reliable - even if errors occur.	

Question	Satisfactory response
What are the organisational guidelines and procedures applicable in assisting the development of data-driven applications?	Yes ☐ No ☐
Answer: As discussed previously in the context of application testing, individual organisational guidelines are often used to help streamline the development of applications. This may include enforcing coding, documentation, version control, and security standards.	Comment:

Question	Satisfactory response
In the context of specifically <i>data-driven</i> applications, some further organisational guidelines may be applicable. These focus specifically on establishing secure and compliant data management. Key procedures include mapping data lifecycles, ensuring ethical privacy compliance, and establishing secure systems. Mapping data lifecycles means clearly defining procedures for where data is stored, how long it is retained, and when it is archived or securely disposed of. This helps ensure the data stored is still relevant, and reduces possible risk involved with keeping records that are unneeded. Further relevant to data-driven applications is consideration of privacy and ethics. When collecting and storing user data, organisations must enforce strict adherence to privacy laws and ethical guidelines to protect this data, prevent information loss, and build customer trust. Finally, when dealing with a data-driven application - and particularly if this data comes from user input - organisations must more seriously consider security risks. Organisations should implement validation rules during data entry, enforce role-based access controls (RBAC), and manage secure, encrypted storage of user data.	

PORTFOLIO

	AI Assisted Learning Permitted	Guided Use
	You may use AI tools to help research, clarify concepts, or check your understanding. However, your final response must demonstrate your own application and comprehension.	

Criteria for assessment	Satisfactory		Comment
	Yes	No	
The following has been submitted for assessment:			
Part 1 - Knowledge questions	☐	☐	
Part 2 – Develop a 2D-Shape Calculator	☐	☐	
Part 3 – Develop a “ToDo” List	☐	☐	
Marking criteria for each product/ document supplied.			
Part 1 - Knowledge questions			
1. Learner correctly discussed class and constructor.	☐	☐	
2. Learner correctly discussed object aggregation	☐	☐	
3. Learner correctly discussed basic programming structures, sequential, condition, loop which is implemented through Object oriented data structure called class	☐	☐	
4. Learner correctly discussed member field and member method in a C# class which contains primitive instance variables	☐	☐	
5. Learner correctly discussed static member field and static member method in a C# class	☐	☐	
6. Learner correctly discussed the debugging and testing approaches and techniques in software development	☐	☐	

Criteria for assessment		Satisfactory Yes No		Comment
7.	Learner correctly discussed software development life cycle	☐	☐	
8.	Learner correctly listed common organizational documentation types and their purposes in SDLC	☐	☐	
9.	Learner correctly discussed small-size application development processes	☐	☐	
10.	Learner correctly discussed the processes, tools and techniques for testing small-size applications	☐	☐	
11.	Learner correctly researched auto test frameworks for the following languages such as C#	☐	☐	
12.	Learner correctly described software test concept "IPO" (Inputs, Process and Outputs)	☐	☐	
13.	Learner correctly explained the following data structure - array, tree	☐	☐	
14.	Learner correctly discussed at least 5 organisational guidelines and procedures applicable to software development and testing frameworks	☐	☐	
15.	Learner correctly discussed software defect management process	☐	☐	
16.	Learner correctly discussed relational database management systems, functions and tools	☐	☐	
17.	Learner correctly discussed the concept of application programming interfaces.	☐	☐	
18.	Learner researched and discussed at least 3 different data providers and vendors	☐	☐	
19.	Learner correctly discussed the principles of Structured Query Language (SQL)	☐	☐	
20.	Learner correctly discussed the organisational guidelines and procedures applicable in assisting the development of data-driven applications	☐	☐	
Marking criteria for each product/ document supplied.				
Part 2 – Develop a 2D-Shape Calculator				
Learner read the project brief for this application, review and clarify user requirements with user. And list all user requirements using bullet points.		☐	☐	
Learner plan and determine the class hierarchy (inheritance tree) and numerical computations are done as per user requirements		☐	☐	
1.3. Learner plan and determine the member fields and methods for all classes.		☐	☐	
Learner explained "inheritance", "polymorphism" and "overriding methods".		☐	☐	
Learner implemented all classes according to the design above using an OOP programming language (e.g. C#).		☐	☐	
Learner implemented two versions of constructors for all classes with member fields and ensure at least implemented one sequence, one selection and two iteration constructs and two arrays of primitive data types in the application.		☐	☐	

Criteria for assessment	Satisfactory		Comment
	Yes	No	
Learner implemented the console user interface according to the requirements which consists of at least 3 coding conventions.	☐	☐	
Marking criteria for each product/ document supplied. Part 3 – Develop Data-driven Applications			
Learner read the project brief for this application, review and clarify user requirements with user. And list all user requirements using bullet points.	☐	☐	
Learner plan and determine the class hierarchy (inheritance tree) which embeds the object-oriented features to satisfy user requirements.	☐	☐	
Learner plan and determine the member fields and methods for all classes.	☐	☐	
Learner explained “inheritance”, “polymorphism” and “overriding methods”.	☐	☐	
Learner implemented all classes according to the design above using an OOP programming language (e.g. C#).	☐	☐	
Learner implemented two versions of constructors for all classes with member fields	☐	☐	
Learner implemented the console user interface according to the requirements	☐	☐	
Learner research disconnected data management strategy	☐	☐	
Learner read and write files using programming language	☐	☐	

Supporting document: Develop a 2D-Shapes Calculator

Part 2 - Develop a 2D-Shapes Calculator

Development requirements

Here are the requirements for the "2D-Shapes Calculator" program.

You are required to

- Design and develop a software program that meets the above requirements
- Develop a test plan that covers the requirements using provided template
- Execute the test plan to verify the program
- Generate a test report using provided template

You are also required to follow the questions in the step 1 and step 2 below to finish all the tasks and answer all the questions.

Development environment

- Visual studio
- SQLite DB-browser

Organisational guidelines

- C# Coding Conventions - <https://docs.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>
- Software test principles and guidelines. See attached file below.

Step 1 – Software design & development

In this step, you are required design and develop the application using a selected programming language (e.g. C#).

1.1. Read the project brief for this application, review and clarify user requirements with user. And list all user requirements using bullet points.

Answer:

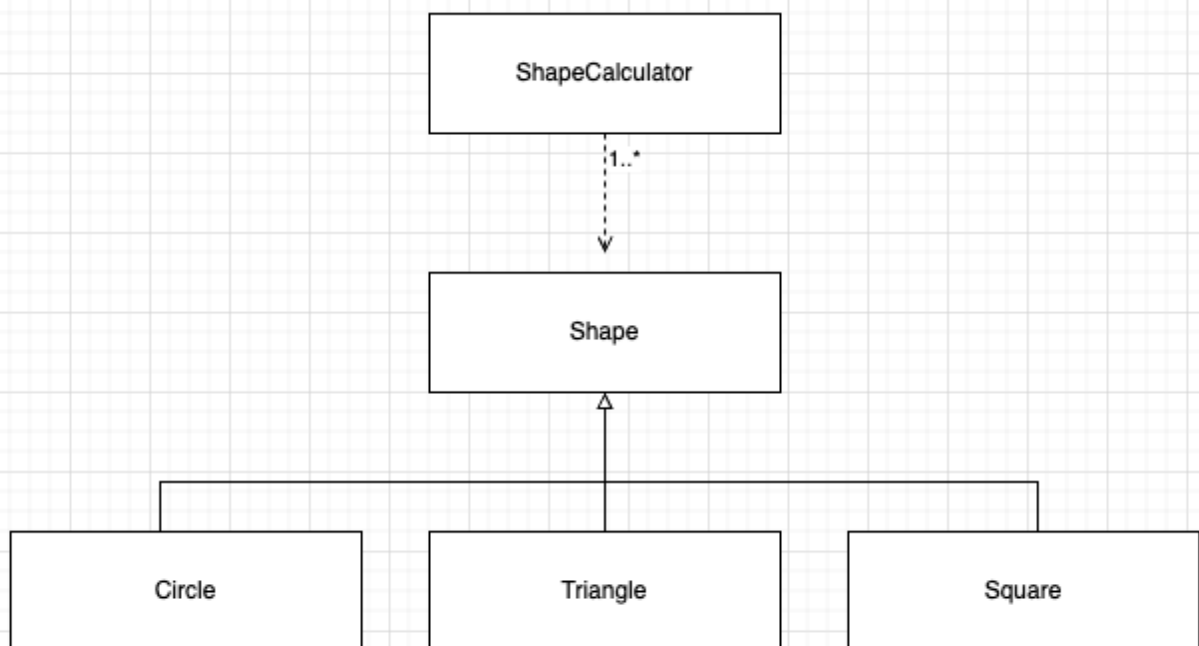
The goal of this project is to develop a shape calculator. The calculator should use a console interface to allow the user to select a shape, input values, and select a calculation based on those values.

User requirements:

- Calculator uses console interface
- User can navigate application with the keyboard
- User can exit the application with the keyboard
- User can make meaningful selections
- User can choose a shape from a provided list
- User can input values where required
- User can select a calculation to be done
- Calculation should complete accurately
- If invalid input is provided, an informative error message should be provided

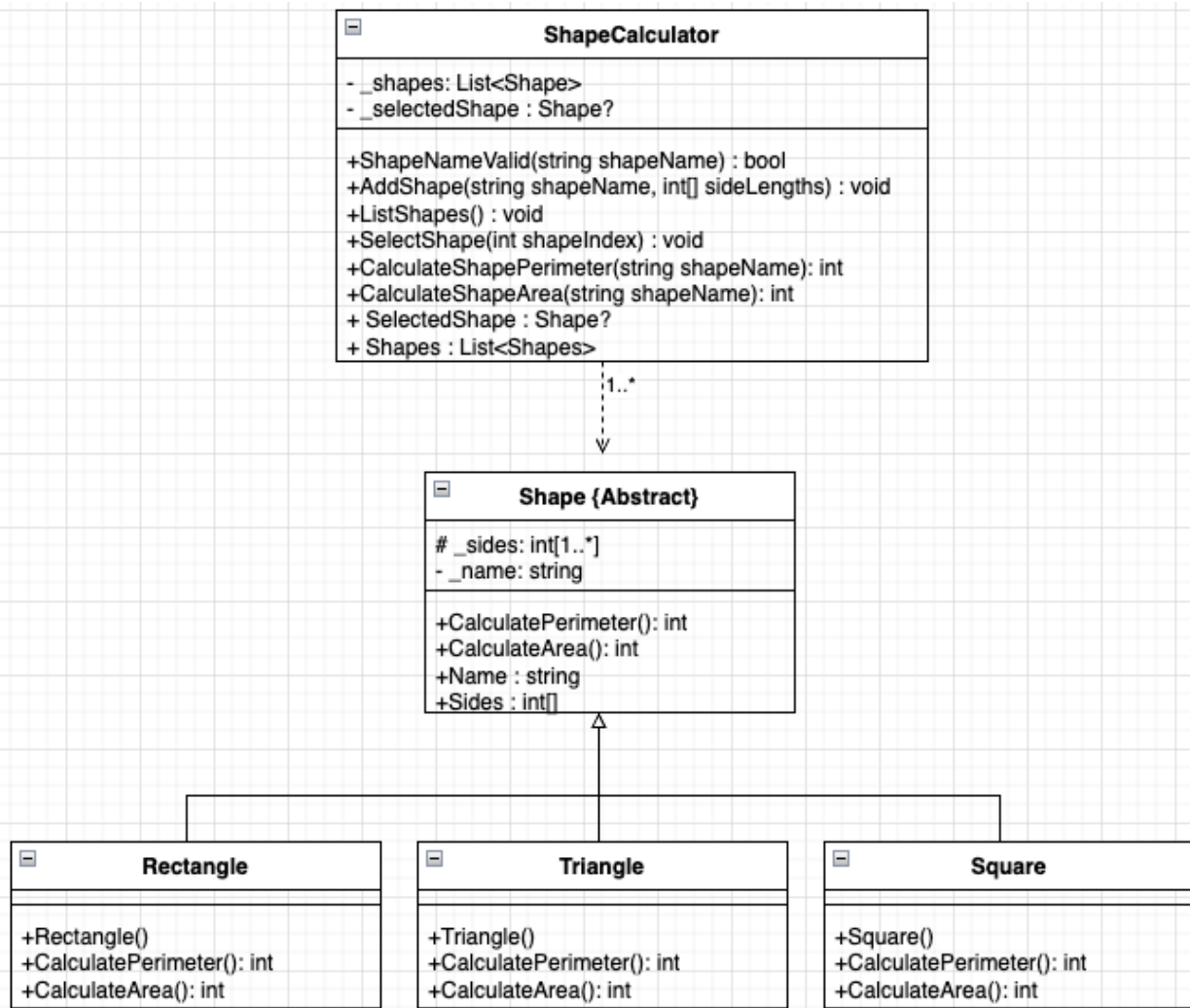
1.2. Plan and determine the class hierarchy (inheritance tree) to satisfy user requirements. You are required to use the following notation to represent the inheritance relationship.

Answer:



1.3. Plan and determine the member fields and methods for **each** class documented in question 2 above. You are required to use the following notation to represent a class.

Answer:



1.4. In terms of object-oriented programming, explain “inheritance”, “polymorphism” and “overriding methods”. You may explain the three terms by using an example.

Answer:

Terms	Your answers
inheritance	Inheritance refers to when a child class inherits from a parent class. This means the child class also contains the methods and fields of the parent class, but may change them and have additional information. For example, a Car class may inherit from a Vehicle class, where the Vehicle class has a field for “wheelsNum” and a method for “TurnOn”, which the Car class also contains and uses.
polymorphism	Polymorphism refers to when a child class is provided when a parent class is expected. This allows flexibility and extendability in programming. An example of this may be a Driver class who has a “Drive(Vehicle currentVehicle)” method. Within this method, the class may require a Vehicle object and call its TurnOn() method. It may and call this method by implementing currentVehicle.TurnOn(). However, any child of Vehicle can be provided - a Car object, or any other class that inherits from Vehicle. They will all implement the TurnOn() method, so when the Driver’s Drive(Vehicle currentVehicle) method is called, which child is provided does not matter.
overriding methods	Overriding methods refers to when a child class changes the behaviour of a parent’s method. The logic within the method can be changed and adapted to suit the child’s goals, though the signature must be the exact same as the parents. This means keeping the exact same method name, parameters, and return type, and the overridden method cannot have a more restrictive

access modifier than its parent. An example of this may be within the Car's implementation of Vehicle's TurnOn() method. In Vehicle's TurnOn() method, a line saying "The vehicle was turned on!" might be printed to the console. However, this method could be overridden in the Car class, where when called, a line saying "The engine starts and the car turns on!" might be shown.

1.5 Implement all classes according to the design above using an OOP programming language (e.g. C#, in Visual Studio IDE). You need to make sure:

- Inheritance and polymorphism (overriding methods) have been correctly implemented.
- Code conventions are followed
- Codes are fully documented using comments

Provide a screenshot of the codes for each class definition.

Answer:

ShapeCalculator

```
1 namespace ShapeCalculator;
2 /*
3  * File: ShapeCalculator.cs
4  * Project: ShapeCalculator
5  * Author: Finnley Newnham
6  * Date: 6 July 2026
7  *
8  * Purpose:
9  * Allows the creation, storage and calculation of shapes.
10  *
11  * The class can create a shape from its name and side lengths, store it in a collection,
12  * and calculate its area or perimeter.
13  */
14
15 /// <summary>
16 /// Creates, stores, finds and performs calculations for shapes.
17 /// </summary>
18 You, 21 hours ago | 1 author (You) | Windsurf: Refactor | Explain
19 public sealed class ShapeCalculator
20 {
21     private List<Shape> _shapes = [];
22     private Shape? _selectedShape;
23
24     /// <summary>
25     /// Creates a supported shape and adds it to the calculator's collection.
26     /// </summary>
27     /// <param name="shapeName">
28     /// The shape type. Supported values are Rectangle, Triangle and Square.
29     /// </param>
30     /// <exception cref="ArgumentException">
31     /// Thrown when the shape name is unsupported.
32     /// </exception>
33     /// <returns>The name of the shape that was chosen.</returns>
34     Windsurf: Refactor | Explain | X
35     public bool ShapeNameValid(string shapeName)
36     {
37         // Ensure values are not null or whitespace
38         ArgumentException.ThrowIfNullOrWhiteSpace(shapeName);
39
40         // A case-insensitive comparison cause Rectangle & rEcTaNgLe should both work
41         shapeName = shapeName.Trim().ToLowerInvariant();
42
43         if (shapeName != "rectangle" && shapeName != "triangle" && shapeName != "square")
44         {
45             throw new ArgumentException($"The shape name '{shapeName}' is not supported, please choose Rectangle, Square
```

```

43         throw new ArgumentException($"The shape name '{shapeName}' is not supported, please choose Rectangi
44     }
45     else
46     {
47         return true; // The shape name is valid and supported. Yippee!
48     }
49 }
50
51 /// <summary>
52 /// Adds a shape to the calculator's collection using its name and side lengths.
53 /// </summary>
54 /// <param name="shapeName">The name of the shape to add.</param>
55 /// <param name="sideLengths">An array of the shape's side lengths.</param>
56 public void AddShape(string shapeName, int[] sideLengths)
57 {
58     // Bulletproofing
59     ArgumentException.ThrowIfNullOrWhiteSpace(shapeName);
60     ArgumentNullException.ThrowIfNull(sideLengths);
61
62     Shape shape;
63
64     switch (shapeName.Trim().ToLowerInvariant()) // Clean the name again
65     {
66         case "rectangle":
67             shape = new Rectangle(sideLengths);
68             break;
69         case "triangle":
70             shape = new Triangle(sideLengths);
71             break;
72         case "square":
73             shape = new Square(sideLengths);
74             break;
75         default:
76             throw new ArgumentException($"The shape name '{shapeName}' is not supported, please choose Rectangle, Square
77     }
78
79     _shapes.Add(shape);
80     Console.WriteLine($"Successfully added a {shape.Name} with side length(s): {string.Join(", ", shape.Sides)}");
81
82
83     /// <summary>
84     /// Lists all the shapes to the console, including their names and side lengths.

```

```

83     /// <summary>
84     /// Lists all the shapes to the console, including their names and side lengths.
85     /// </summary>
86     public void ListShapes()
87     {
88         if (_shapes.Count == 0)
89         {
90             Console.WriteLine("No shapes have been added yet.");
91             return;
92         }
93
94         Console.WriteLine("Shapes in the calculator:");
95         int i = 1;
96         foreach (var shape in _shapes)
97         {
98             Console.WriteLine($"{i}. {shape.Name} - Side length(s): {string.Join(", ", shape.Sides)}");
99             i++;
100         }
101     }
102
103     /// <summary>
104     /// Selects a shape from the calculator's list by index +1.
105     /// Designed to be used with the ListShapes() method, which lists shapes to the user starting from 1.
106     /// </summary>
107     /// <param name="shapeIndex">The index of the shape to select.</param>
108     public void SelectShape(int shapeIndex)
109     {
110         if (shapeIndex < 1 || shapeIndex > _shapes.Count)
111         {
112             throw new ArgumentOutOfRangeException("Shape index is out of range.");
113         }
114
115         _selectedShape = _shapes[shapeIndex - 1];
116     }
117
118     /// <summary>
119     /// Calculates the perimeter of the selected shape.
120     /// </summary>
121     /// <returns>The perimeter in whole units.</returns>
122     public int CalculateShapePerimeter()
123     {
124

```

```

123 public int CalculateShapePerimeter()
124 {
125     if (_selectedShape == null)
126     {
127         throw new InvalidOperationException("No shape has been selected. Please select a shape before calculating perimeter.");
128     }
129     return _selectedShape.CalculatePerimeter();
130 }
131
132 /// <summary>
133 /// Calculates the area of the selected shape.
134 /// </summary>
135 /// <returns>The area in whole square units.</returns>
136 public int CalculateShapeArea()
137 {
138     if (_selectedShape == null)
139     {
140         throw new InvalidOperationException("No shape has been selected. Please select a shape before calculating area.");
141     }
142     return _selectedShape.CalculateArea();
143 }
144
145 /// <summary>
146 /// Gets the currently selected shape or null if one is not selected.
147 /// </summary>
148 public Shape? SelectedShape => _selectedShape;
149
150 /// <summary>
151 /// Gets the list of all shapes in the calculator.
152 /// </summary>
153 public List<Shape> Shapes => _shapes;
154 }
155

```

Shape

```
You, 21 hours ago | 1 author (You)
1 namespace ShapeCalculator;
2 /*
3  * File: Shape.cs
4  * Project: ShapeCalculator
5  * Author: Finnley Newnham
6  * Date: 6 July 2026
7  *
8  * Purpose:
9  * Defines the abstract Shape base class used by all concrete shape types.
10 * The class stores common shape information and defines the area and
11 * perimeter methods that each derived shape class must implement.
12 */
13
14 /// <summary>
15 /// Defines the information and behaviour shared by every shape.
16 /// </summary>
17 /// <remarks>
18 /// This is an abstract class because a generic shape does not have enough
19 /// information to calculate an area or perimeter. Derived classes provide
20 /// those calculations for their specific shape, along with the shape name.
21 /// </remarks>
You, 21 hours ago | 1 author (You) | Windsurf: Refactor | Explain
22 public abstract class Shape
23 {
24     protected int[] _sides;
25     private string _name;
26
27     /// <summary>
28     /// Initialises a new instance of the <see cref="Shape"/> class.
29     /// </summary>
30     /// <param name="name">The name used to identify the type of shape.</param>
31     /// <param name="sides">The lengths of all sides belonging to the shape.</param>
32     /// <exception cref="ArgumentException">
33     /// Thrown when <paramref name="name"/> is blank or no side lengths are supplied.
34     /// </exception>
35     /// <exception cref="ArgumentOutOfRangeException">
36     /// Thrown when a side length is zero or negative.
37     /// </exception>
38
39     Windsurf: Refactor | Explain | Generate Documentation | X
40     protected Shape(string name, int[] sides)
41     {
42         ArgumentException.ThrowIfNullOrWhiteSpace(name); // Shapes must have a name!
43         _name = name;
```

```

42     _name = name;
43
44     if (sides is null || sides.Length == 0)
45     {
46         throw new ArgumentException("At least one side length must be supplied.");
47     }
48
49     if (sides.Any(side => side <= 0)) // Lambda expression checks for any side length that is zero or negative
50     {
51         throw new ArgumentOutOfRangeException(nameof(sides), "Side lengths must be greater than zero.");
52     }
53
54     // Copy the array so Shape's _sides is not a reference
55     _sides = (int[])sides.Clone();
56 }
57
58 /// <summary>
59 /// Calculates the total distance around the outside of the shape.
60 /// </summary>
61 /// <returns>The perimeter in whole units.</returns>
62 public abstract int CalculatePerimeter();
63
64 /// <summary>
65 /// Calculates the area inside the shape.
66 /// </summary>
67 /// <returns>The area in whole square units.</returns>
68 public abstract int CalculateArea();
69
70 /// <summary>
71 /// Gets the name of the shape.
72 /// </summary>
73 public string Name => _name;
74
75 /// <summary>
76 /// Gets the sides of the shape.
77 /// </summary>
78 public int[] Sides => _sides;
79
80

```

Square

```
namespace ShapeCalculator;
/*
 * File: Square.cs
 * Project: ShapeCalculator
 * Author: Finnley Newnham
 * Date: 6 July 2026
 *
 * Purpose:
 * Inherits from the Shape base class to provide the area and perimeter calculations for a square.
 */

/// <summary>
/// Represents a square whose four sides have the same length.
/// </summary>
You, 21 hours ago | 1 author (You) | Windsurf: Refactor | Explain
public class Square : Shape
{
    /// <summary>
    /// Initialises a new square whose sides all have the length of 1.
    /// </summary>
    Windsurf: Refactor | Explain | X
    public Square() : this([1])
    {
    }

    /// <summary>
    /// Initialises a new square using one given length for all four sides.
    /// </summary>
    You, 21 hours ago * Completed Shape Calculator ...
    /// <param name="sideLengths">
    /// An array containing exactly one value – the length of every side.
    /// </param>
    /// <exception cref="ArgumentException">
    /// Thrown when exactly one side length is not supplied.
    /// </exception>
    Windsurf: Refactor | Explain | X
    public Square(int[] sideLengths) : base("square", sideLengths)
    {
        // A square only needs one supplied value because all four sides are equal!
        if (sideLengths.Length != 1)
        {
            throw new ArgumentException("A square requires exactly one side length, as all four lengths are the same.");
        }
    }

    /// <inheritdoc/>
    Windsurf: Refactor | Explain | X
    public override int CalculatePerimeter()
    {
        return _sides[0] * 4; // A square has four equal sides, so multiply the single side length by 4
    }

    /// <inheritdoc/>
    Windsurf: Refactor | Explain | X
    public override int CalculateArea()
    {
        return _sides[0] * _sides[0]; // A square's area is the length of one side squared
    }
}
```

Triangle

```

1 namespace ShapeCalculator;
2 /*
3  * File: Triangle.cs
4  * Project: ShapeCalculator
5  * Author: Finnley Newnham
6  * Date: 6 July 2026
7  *
8  * Purpose:
9  * Inherits from the Shape base class to provide the area and perimeter calculations for a triangle.
10 */
11
12 /// <summary>
13 /// Represents a three-sided triangle.
14 /// </summary>
15 You, 21 hours ago | 1 author (You) | Windsurf: Refactor | Explain
16 public class Triangle : Shape
17 {
18     /// <summary>
19     /// Initialises a new triangle with three equal sides of 1.
20     /// </summary>
21     Windsurf: Refactor | Explain | X
22     public Triangle() : this([1, 1, 1])
23     {
24     }
25
26     /// <summary>
27     /// Initialises a new triangle using three side lengths.
28     /// </summary>
29     /// <param name="sideLengths">The three side lengths of the triangle.</param>
30     /// <exception cref="ArgumentException">
31     /// Thrown when exactly three side lengths are not supplied.
32     /// </exception>
33     Windsurf: Refactor | Explain | X
34     public Triangle(int[] sideLengths) : base("triangle", sideLengths)
35     {
36         // A triangle must have exactly three side lengths
37         if (sideLengths.Length != 3)
38         {
39             throw new ArgumentException("A triangle requires exactly three side lengths.");
40         }
41     }
42 }

```

```

31 public Triangle(int[] sideLengths) : base("triangle", sideLengths)
32 {
33     // A triangle must have exactly three side lengths
34     if (sideLengths.Length != 3)
35     {
36         throw new ArgumentException("A triangle requires exactly three side lengths.");
37     }
38     if (sideLengths[0] + sideLengths[1] <= sideLengths[2] ||
39         sideLengths[0] + sideLengths[2] <= sideLengths[1] ||
40         sideLengths[1] + sideLengths[2] <= sideLengths[0])
41     {
42         throw new ArgumentException("The supplied side lengths do not form a valid triangle.");
43     }
44 }
45
46 /// <inheritdoc/>
47 Windsurf: Refactor | Explain | X
48 public override int CalculatePerimeter()
49 {
50     return _sides.Sum(); // A triangle's perimeter is the sum of its three side lengths
51 }
52
53 /// <inheritdoc/>
54 Windsurf: Refactor | Explain | X
55 public override int CalculateArea()
56 {
57     return (int)Math.Sqrt(
58         CalculatePerimeter() / 2.0 *
59         (CalculatePerimeter() / 2.0 - _sides[0]) *
60         (CalculatePerimeter() / 2.0 - _sides[1]) *
61         (CalculatePerimeter() / 2.0 - _sides[2])); // Heron's formula for triangle area given all three side lengths
62 }

```

Rectangle

```

1 namespace ShapeCalculator;
2 /*
3  * File: Rectangle.cs
4  * Project: ShapeCalculator
5  * Author: Finnley Newnham
6  * Date: 6 July 2026
7  *
8  * Purpose:
9  * Inherits from the Shape base class to provide the area and perimeter calculations for a rectangle.
10 */
11
12 /// <summary>
13 /// Represents a four-sided rectangle.
14 /// </summary>
15 public class Rectangle : Shape
16 {
17     /// <summary>
18     /// Initialises a new rectangle with a length of 1 and a width of 2.
19     /// </summary>
20     public Rectangle() : this([1, 2])
21     {
22     }
23
24     /// <summary>
25     /// Initialises a new rectangle using its length and width.
26     /// </summary>
27     /// <param name="sideLengths">
28     /// Two values containing the rectangle's length and width.
29     /// </param>
30     /// <exception cref="ArgumentException">
31     /// Thrown when exactly two side lengths are not supplied, or both side lengths are the same.
32     /// </exception>
33     public Rectangle(int[] sideLengths) : base("rectangle", sideLengths)
34     {
35         // A rectangle must have one length and one width
36         if (sideLengths.Length != 2)
37         {
38             throw new ArgumentException("A rectangle requires exactly two lengths: its length and width.");
39         }
40     }
41 }

```

```

42     // Equal dimensions would make this a square, rather than a rectangle
43     if (sideLengths[0] == sideLengths[1])
44     {
45         throw new ArgumentException("A rectangle's length and width must be different. Use a Square for equal lengths.");
46     }
47
48     /// <inheritdoc>
49     public override int CalculatePerimeter()
50     {
51         return (_sides[0] + _sides[1]) * 2; // A rectangle's perimeter is the sum of its length and width, multiplied by 2
52     }
53
54     /// <inheritdoc>
55     public override int CalculateArea()
56     {
57         return _sides[0] * _sides[1]; // A rectangle's area is its length multiplied by its width
58     }
59 }
60
61

```

- ☒ I have followed code conversions
- ☒ I have fully documented using comments

1.6 Implement two versions of constructors for all classes with member fields.

- One constructor accepts zero parameter and initialize each member fields to a reasonable default value.
- The other constructor accepts a number of parameters for initializing all member fields.

Provide a screenshot of the two constructors for each class.

Answer:

Class Name	Constructor #1 – without parameter Screenshot
Square	Without parameter <pre>public Square() : this([1]) { }</pre> With parameter <pre>public Square(int[] sideLengths) : base("square", sideLengths) { // A square only needs one supplied value because all four sides are equal! if (sideLengths.Length != 1) { throw new ArgumentException("A square requires exactly one side length, as all four lengths are the same."); } }</pre>
Rectangle	Without parameter <pre>public Rectangle() : this([1, 2]) { }</pre> With parameter <pre>public Rectangle(int[] sideLengths) : base("rectangle", sideLengths) { // A rectangle must have one length and one width if (sideLengths.Length != 2) { throw new ArgumentException("A rectangle requires exactly two lengths: its length and width."); } // Equal dimensions would make this a square, rather than a rectangle if (sideLengths[0] == sideLengths[1]) { throw new ArgumentException("A rectangle's length and width must be different. Use a Square for equal dimensions."); } }</pre>
Triangle	Without parameter <pre>public Triangle() : this([1, 1, 1]) { }</pre>

With parameter

```
public Triangle(int[] sideLengths) : base("triangle", sideLengths)
{
    // A triangle must have exactly three side lengths
    if (sideLengths.Length != 3)
    {
        throw new ArgumentException("A triangle requires exactly three side lengths.");
    }
    if (sideLengths[0] + sideLengths[1] <= sideLengths[2] ||
        sideLengths[0] + sideLengths[2] <= sideLengths[1] ||
        sideLengths[1] + sideLengths[2] <= sideLengths[0])
    {
        throw new ArgumentException("The supplied side lengths do not form a valid triangle.");
    }
}
```

1.7 Implement the console user interface according to the requirements. Provide a screenshot of the codes.

You need to make sure:

- Code conventions are followed
- Codes are fully documented using comments

Answer:

Your screenshot

```
namespace ShapeCalculator;
/*
 * File: Program.cs
 * Project: ShapeCalculator
 * Author: Finnley Newnham
 * Date: 6 July 2026
 *
 * Purpose:
 * Allows the use of a shape calculator through a command-line interface.
 *
 * Use dotnet run --project ShapeCalculator/ShapeCalculator/ShapeCalculator.csproj to run.
 */

/// <summary>
/// Provides the command-line entry point for the application and user interface with flavour text.
/// </summary>
You, 22 hours ago | 1 author (You) | Windsurf: Refactor | Explain
internal static class Program
{
    /// <summary>
    /// Starts the Shape Calculator application.
    /// </summary>
    Windsurf: Refactor | Explain | X
    private static void Main()
    {
        ShapeCalculator shapeCalculator = new ShapeCalculator();
        Console.WriteLine("Welcome to the Shape Calculator!");

        string selection = "";
        while (selection != "")
        {
            if (shapeCalculator.SelectedShape != null)
            {
                Console.WriteLine($"{\n ==> Selected shape: {shapeCalculator.SelectedShape.Name} - {string.Join(", ", shapeCalculator.SelectedShape.Sides)}");
            }
            Console.WriteLine("\n >>> Please select an option:");
            Console.WriteLine("1. Add a shape");
            Console.WriteLine("2. List shapes");
            Console.WriteLine("3. Select a shape");
        }
    }
}
```

```

37 Console.WriteLine("3. Select a shape");
38 if (shapeCalculator.SelectedShape != null)
39 {
40     Console.WriteLine("4. Calculate perimeter");
41     Console.WriteLine("5. Calculate area");
42 }
43 Console.WriteLine("\n --- Or, enter '' at any time to quit the application.\n\n");
44
45 selection = Console.ReadLine()?.Trim().ToLowerInvariant() ?? ""; // Clean user input and handle nulls
46
47 switch (selection)
48 {
49     case "1":
50         Console.WriteLine("Please enter the name of the shape to add.\n > Supported shapes are: square, triangle, and rectangle");
51         string newShapeName = Console.ReadLine() ?? "";
52
53         try
54         {
55             if (shapeCalculator.ShapeNameValid(newShapeName))
56             {
57                 Console.WriteLine($"Great! Lets add a {newShapeName} to the calculator. Please enter the side lengths as whole numbers, separated by commas.");
58                 Console.WriteLine($"! As a reminder, a square requires one side length, a rectangle requires a width and height, and a triangle requires two side lengths.");
59                 Console.WriteLine("Please separate the side lengths with commas, and do not include any letters or symbols.");
60                 string? sideLengthsInput = Console.ReadLine()?.Trim();
61
62                 if (!string.IsNullOrEmpty(sideLengthsInput))
63                 {
64                     // Attempt to split the input based on commas, trim whitespace, and convert to integers
65                     int[] sideLengths = sideLengthsInput.Split(',').Select(s => int.Parse(s.Trim())).ToArray();
66                     shapeCalculator.AddShape(newShapeName, sideLengths);
67                     Console.WriteLine($"The {newShapeName} was added successfully.");
68                 }
69             }
70             else
71             {
72                 shapeCalculator.AddShape(newShapeName);
73             }
74         }
75         catch (FormatException)
76         {
77             Console.WriteLine("Invalid input for side lengths. Please enter whole numbers separated by commas.");
78         }
79         catch (ArgumentException exception)
80         {
81             // Shape validation errors are displayed before control returns to the menu.
82             Console.WriteLine($"The shape could not be added: {exception.Message}");
83         }
84         break;
85     case "2":
86         shapeCalculator.ListShapes();
87         break;
88     case "3":
89         if (shapeCalculator.Shapes.Count == 0)
90         {
91             Console.WriteLine("No shapes have been added yet. Please add a shape first.");
92             break;
93         }
94         shapeCalculator.ListShapes();
95         Console.WriteLine("Please enter the number of the shape to select.");
96
97         string? shapeSelectionInput = Console.ReadLine();
98
99         // TryParse checks that the user's input can be converted to an int
100         if (int.TryParse(shapeSelectionInput, out int selectedIndex))
101         {
102             try

```

```

102         try
103         {
104             shapeCalculator.SelectShape(selectionIndex);
105             Console.WriteLine($"You selected {shapeCalculator.SelectedShape?.Name}.");
106         }
107         catch (ArgumentOutOfRangeException)
108         {
109             Console.WriteLine("That shape number does not exist. Please choose a number from the list.");
110         }
111     }
112     else
113     {
114         Console.WriteLine("Invalid input. Please enter a whole number.");
115     }
116     break;
117 case "4":
118     if (shapeCalculator.SelectedShape != null)
119     {
120         Console.WriteLine($"The perimeter of the selected shape is: {shapeCalculator.CalculateShapePerimeter()}");
121     }
122     break;
123 case "5":
124     if (shapeCalculator.SelectedShape != null)
125     {
126         Console.WriteLine($"The area of the selected shape is: {shapeCalculator.CalculateShapeArea()}");
127     }
128     break;
129 case "":
130     Console.WriteLine("Thank you for using the Shape Calculator!");
131     break;
132 default:
133     Console.WriteLine("Invalid option. Please try again.");
134     break;
135 }
136 }
137 }
138 You, 22 hours ago • Completed Shape Calculator
139

```

```

Welcome to the Shape Calculator!

>>> Please select an option:
1. Add a shape
2. List shapes
3. Select a shape

--- Or, enter '' at any time to quit the application.

1
Please enter the name of the shape to add.
> Supported shapes are: square, triangle, and rectangle
2
The shape could not be added: The shape name '2' is not supported, please choose Rectangle, Square or Triangle.

>>> Please select an option:
1. Add a shape
2. List shapes
3. Select a shape

--- Or, enter '' at any time to quit the application.

1
Please enter the name of the shape to add.
> Supported shapes are: square, triangle, and rectangle
triangle
Great! Lets add a triangle to the calculator. Please enter the side lengths as whole numbers, separated by commas.

! As a reminder, a square requires one side length, a rectangle requires a width and height, and a triangle requires three side lengths.
Please separate the side lengths with commas, and do not include any letters or symbols.
1,2,2
Successfully added a triangle with side length(s): 1, 2, 2

>>> Please select an option:
1. Add a shape
2. List shapes
3. Select a shape

--- Or, enter '' at any time to quit the application.

2
Shapes in the calculator:
1. triangle - Side length(s): 1, 2, 2

>>> Please select an option:
1. Add a shape
2. List shapes
3. Select a shape

--- Or, enter '' at any time to quit the application.

```

- ☒ I have followed code conversions
- ☒ I have fully documented using comments

PART 2 - OBSERVATION

Below are the observation components that the learner is required to demonstrate as part of this assessment. The learner only needs to demonstrate this once. If the learner is unable to demonstrate one or more of the required Observation tasks to a satisfactory level, they will be given two opportunities to demonstrate the unsatisfactory Observation task(s).

Marking criteria: Skills to be observed during this task to the required standard

In this assessment task you will be performing following activities	Date 1		Date 2		Date 3		Comment
	Resubmission if applicable		Resubmission if applicable		Resubmission if applicable		
	Satisfactory		Satisfactory		Satisfactory		
	Yes	No	Yes	No	Yes	No	
Develop a 2D-Shape Calculator							
1.8. Learner implemented polymorphism for code extensibility	☐	☐	☐	☐	☐	☐	

In this assessment task you will be performing following activities	Date 1		Date 2		Date 3		Comment
			Resubmission if applicable		Resubmission if applicable		
	Satisfactory		Satisfactory		Satisfactory		
	Yes	No	Yes	No	Yes	No	
1.9. Learner applied documentation conventions and documented the application according to organisational documentation conventions	☐	☐	☐	☐	☐	☐	
1.10. Learner applied code conventions to organisational requirements	☐	☐	☐	☐	☐	☐	
1.11. Learner communicated relationships between ideas and information, in a style appropriate to the audience and purpose, and selects the vocabulary, grammatical structures and conventions appropriate to the text, in relation to coding, recording outcomes, and documenting activities.	☐	☐	☐	☐	☐	☐	
1.12. Learner selected from, and flexibly applies, mathematical and problem-solving strategies and techniques, in a programming context	☐	☐	☐	☐	☐	☐	
1.13. Learner recognised and followed, explicit and implicit standard and meets expectations associated with own role when developing code that is compliant with standards and guidelines	☐	☐	☐	☐	☐	☐	
1.14. Learner demonstrated a deliberate decision-making method, weighing various alternatives against specific criteria to select the most appropriate data types, operators, and expressions.	☐	☐	☐	☐	☐	☐	
1.15. Learner evaluated the effectiveness of decisions, in terms of how well they meet the stated design specifications	☐	☐	☐	☐	☐	☐	
1.16. Learner used analytical processes to decide on a course of action when debugging	☐	☐	☐	☐	☐	☐	
1.17. Learner utilised features within applications in order to develop software programs that involve mathematical equations.	☐	☐	☐	☐	☐	☐	
1.18. Learner recognised, and used language and symbols, when applying the coding syntax	☐	☐	☐	☐	☐	☐	
1.19. Learner actively identified systems, devices and applications with the potential to meet current and future needs regarding programming	☐	☐	☐	☐	☐	☐	
1.20. Learner reviewed application against user requirements to ensure user requirements are satisfied	☐	☐	☐	☐	☐	☐	
1.21. Learner presented application to user and obtain user acceptance	☐	☐	☐	☐	☐	☐	

Step 2 – Software test

2.1 Identify unit test requirements for each class and list all requirements with bullet points.

Answer:

The following tests cases are detailed in the Test Plan and executed on 07/07/26, with results detailed within the Test Report.

Square

- Test Functional Unit (perimeter)
- Test Functional Unit (area)
- Equivalent partition,
- Stress test
- Boundary test

Rectangle

- Test Functional Unit (perimeter)
- Test Functional Unit (area)
- Equivalent partition,
- Boundary test

Triangle

- Test Functional Unit (perimeter)
- Test Functional Unit (area)
- Equivalent partition,
- Boundary test

2.2 Identify system test requirements for the application and list all requirements with bullet points

Answer:

Console Interface Requirements

- The system shall provide a **command-line (console-based) interface** for user interaction.
- The system shall display clear **menu options and prompts** to guide the user.
- The system shall accept user input via the keyboard.
- The system shall display results and error messages in the console.

Add Shape Requirements

- The system shall allow the user to add a shape predefined list.
- The predefined list of available shapes shall include:
 - Square
 - Rectangle
 - Triangle
- The system shall prevent invalid input outside of the predefined list.
- The system shall prompt the user again if invalid input is entered.
- The system should prompt the user to input side lengths for a new shape
- If invalid inputs are entered, the system should not add the shape

Calculation Requirements

- The system shall allow the user to choose **what to calculate** for the selected shape.
- The calculation options shall include:
 - Perimeter
 - Area
- The system shall only display calculation options relevant to the selected shape - and only if there is a selected shape.
- The system shall reject invalid calculation choices and request valid input.

Input Handling Requirements

- The system shall prompt the user to enter all **required dimensions** for the chosen shape and calculation.
- The system shall validate that all numeric inputs are:
 - Present

- Numeric
- Greater than zero
- Valid for the chosen shape
- The system shall display appropriate error messages for invalid or missing input.
- The system shall allow the user to re-enter values after invalid input.

Workflow and Usability Requirements

- The system shall allow the user to perform **multiple calculations** without restarting the application.
- The system shall allow the user to exit the application gracefully.
- The system shall not crash or terminate unexpectedly during normal operation.

Error Handling Requirements

- The system shall handle invalid menu selections gracefully.
- The system shall handle invalid numeric input without crashing.
- The system shall provide meaningful error messages to the user.

2.3 Discuss and confirm the above development requirements with a **member in software development team**.

Note: The member in software development team can be role played by another student. You are required to discuss the following items in the meeting.

Answer:

Participants/roles	Tester / QA Engineer (Student) – Responsible for designing and executing test cases Software Developer (Role-played by another student) – Responsible for implementing the calculator application Project Representative – Confirmed requirements and testing expectations
Meeting date	07 July 2026
Scope of test	<i>The following scope of testing was agreed upon during the meeting:</i> Included in scope ● Testing of the command-line console interface ● Verification of user ability to: ○ Add a shape ○ List shapes ○ Select a shape ○ Perform calculations with added shape ○ Close program ● Validation of user input (numeric values, positive numbers) ● Verification of correct calculation results ● Error handling for invalid user inputs ● End-to-end system behavior
Types of tests	<i>Unit Testing – To verify individual calculation methods for each shape</i> <i>Functional Testing – To verify the complete application flow from user input to output</i> <i>Boundary and Equivalence Testing – To validate input ranges and representative values</i> <i>Negative Testing – To ensure correct handling of invalid input</i>
Tools	Microsoft Visual Studio Windows Command Line MSTest Framework
Benefits	Microsoft Visual Studio Microsoft Visual Studio provides an integrated development and testing environment that allows developers and testers to write, run, and debug code efficiently. Its built-in testing features and error diagnostics help identify defects early in the development process, improving software quality. Windows Command Line The Windows command line allows realistic testing of the console-based application in the same environment used by end users. This ensures that user input handling, prompts, and outputs function correctly in real execution conditions. MSTest Framework MSTest supports automated unit testing and provides clear test results and

assertions. It enables repeatable, reliable testing of calculation logic and helps ensure that changes to the code do not introduce new defects.

2.4 Identify organisational guidelines and frameworks related to testing. List them all using bullet points

Answer:

Software Development Life Cycle (SDLC) Policy –

- Our company utilises a Kanban framework
- Identified bugs should be added to the relevant development board
- QA must assess technical severity
- QA must assess business priority
- QA must assess effort vs. impact
- Once evaluated, relevant personnel will be assigned to remedying the bug
- QA will assess the implemented solution

Coding Standards and Conventions - A set of rules for writing clean, consistent, and readable code.

- Changes must be well documented
 - Comments
 - XML documentation
 - External documentation
- Changes must implement relevant styles and be linted
- Changes must pass CI/CD checks
- Merges must be reviewed by one senior member of personnel

Testing and Quality Assurance (QA) Procedures - Ensures software is **reliable, functional, and meets user requirements** before release.

- QA team should undergo stress and boundary testing for the fix
- QA team should undergo regular routine system tests to ensure no new bugs have been introduced

2.5 Discuss at least three test-case design techniques and explain how they are applied in your test case design.

Answer:

Test-case design technique #1

Discuss this technique

. Functional Unit Test

Square Perimeter Functional Unit Test

Step 1. My expected output = 20
Step 2. Create an object of Square type overridden
Step 3. Call CalculatePerimeter() and get the actual output
Step 4. Verify expected output and actual output

Test-case design technique #2

Discuss this technique

Equivalence Partitioning

Boundary value test

Triangle Equivalence Partitioning Unit Test

Step 1. My expected output = ArgumentException
Step 2. Create an object of Triangle with invalid side lengths
Step 3. Verify expected output and actual output

Triangle Boundary Unit Test

Step 1. My expected output = ArgumentException
Step 2. Create an object of Triangle with side lengths of 0
Step 3. Verify expected output and actual output

Test-case design technique #3

Discuss this technique

Stress test

Rectangle Stress Unit Test

Step 1. My expected output = 2000000000

Step 2. Create an object of Rectangle with side lengths [20000, 10000]

Step 3. Call CalculateArea() and get the actual output

Step 4. Verify expected output and actual output

2.6 Develop test plan and test cases using provided template.

You are required to apply the test-case design techniques discussed above and develop TWO test cases with EACH technique.

Write the submitted test plan file name in the answer box below.

Answer:

Unit_Test_Plan.pdf

System_Test_Plan.pdf

2.7 Develop unit test procedures using an auto testing framework (e.g. Visual Studio Unit Testing). Provide a screenshot of the unit test codes (if the codes are too long, screenshot 2 test case for this question and your teacher will check all the codes in your code submission).

Answer:

Further code within [ShapeCalculatorTests.cs](#)

```
// Perimeter unit functional tests -----  
Windsurf: Refactor | Explain | X  
[TestMethod]  
public void TestSquarePerimeter()  
{  
    int expectedOutput = 20;  
    Square s = new Square([5]);  
    int actualOutput = s.CalculatePerimeter();  
    Assert.AreEqual(expectedOutput, actualOutput);  
}
```

```
Windsurf: Refactor | Explain | Generate Documentation | X  
[TestMethod]  
public void TestRectanglePerimeter()  
{  
    int expectedOutput = 16;  
    Rectangle r = new Rectangle([5, 3]);  
    int actualOutput = r.CalculatePerimeter();  
    Assert.AreEqual(expectedOutput, actualOutput);  
}
```

```
Test summary: total: 17, failed: 0, succeeded: 17, skipped: 0, duration: 0.5s  
Build succeeded in 1.6s
```

2.8 Perform manual test according to system test plan, screenshot the test results (for at least two test cases).

Answer:

Test Case ID: TC_001

Objective: Add a square with default values.

```
>>> Please select an option:
1. Add a shape
2. List shapes
3. Select a shape

--- Or, enter '' at any time to quit the application.

1
Please enter the name of the shape to add.
> Supported shapes are: square, triangle, and rectangle
square
Great! Lets add a square to the calculator. Please enter the side lengths as whole numbers, separated by commas.

! As a reminder, a square requires one side length, a rectangle requires a width and height, and a triangle requires three side lengths.
Please separate the side lengths with commas, and do not include any letters or symbols.

Successfully added a square with DEFAULT side length(s): 1
```

Test Case ID: TC_002

Objective: Calculate default square's perimeter

```
=> Selected shape: square - 1

>>> Please select an option:
1. Add a shape
2. List shapes
3. Select a shape
4. Calculate perimeter
5. Calculate area

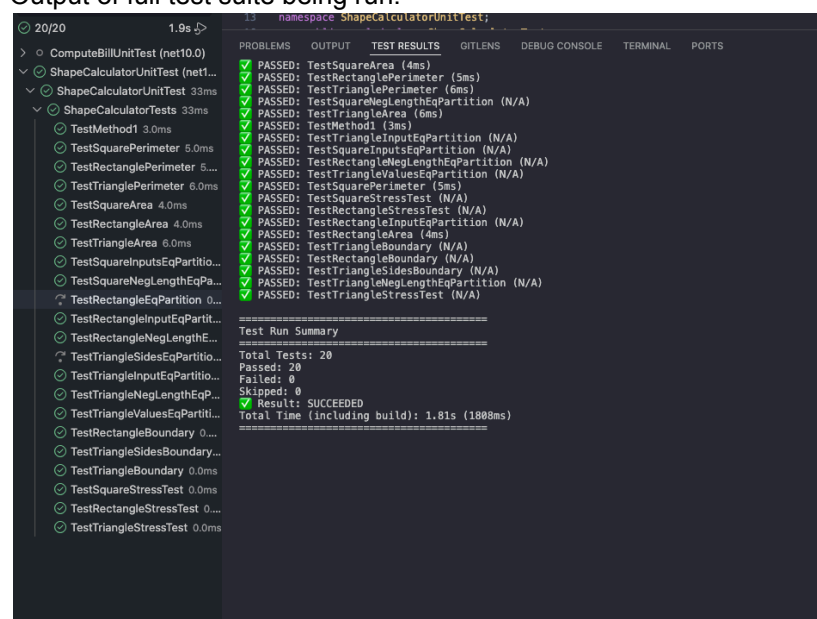
--- Or, enter '' at any time to quit the application.

4
The perimeter of the selected shape is: 4
```

2.9 Perform auto test according to unit test plan, screenshot the test results (for at least two test cases).

Answer:

Output of full test suite being run:



2.10 Discuss and confirm the above **test results with a member in software development team.**

Note: The member in software development team and be role played by another learner.

Answer:

Participants/roles	Project Engineer
Meeting date	07/07/26
Meeting minutes	- Test suite was run with every test passing - System testing was done with member of the software development team - Various shapes with varying side lengths were added, including each shapes's parameterised and parameterless constructors - Shapes were listed - Perimeter calculations were performed and verified on these shapes - Area calculations were performed and verified on these shapes - Invalid input for shape name and side lengths was provided - Attempts to list or perform calculations on shapes when none exist was attempted - Identified bug with Triangle not verifying mathematically invalid inputs

2.11 Produce and save test reports using provided template.

Write the submitted test report file name in the answer box below.

Answer:

System_Test_Report.pdf

Unit_Test_Report.pdf

2.12 Manage defects and review software, amending further defects. You are required to record how did you managed defect for at least one issue based on the test results using the template below.

Answer:

Issue description

Test Case ID

ST-TRI-01

Defect Title / Description

Incorrect handling of mathematically impossible values input for the triangle shape

Steps to Reproduce

1. Launch the console-based shape calculator.
2. Select **Add Shape**
3. Enter "triangle"
4. Enter mathematically impossible values as side lengths (*ie. 1, 2, 3*)
5. Select **List Shapes**
6. Observe successful addition of the invalid triangle shape

Expected Result

- The system should display an appropriate error message indicating invalid input.
- The system should not allow a mathematically impossible triangle to be added.
- The system should return to the menu or request re-entry of valid input.

Actual Result

- The system added the mathematically invalid triangle and allowed calculations
- The system was unable to produce meaningful calculations based on the mathematically impossible triangle

Root Cause Analysis

The validation logic for the Triangle class within the constructor was insufficient. The system checked for the right number of side, a positive and great-than-zero-value, but did not validate the mathematical possibility of the shape.

Defect Fix / Amendment

A conditional validation check must be added within the constructor:

- If [not a valid triangle]:
 - Displays an error message to the user
 - Returns a safe error value
 - Returns to menu

Code Amendment (Summary)

Input validation logic was implemented within the constructor to validate the mathematical possibility of the triangle. Unit tests were added to cover this.

```
if (sideLengths[0] + sideLengths[1] <= sideLengths[2] ||
    sideLengths[0] + sideLengths[2] <= sideLengths[1] ||
    sideLengths[1] + sideLengths[2] <= sideLengths[0])
{
    throw new ArgumentException("The supplied side lengths do not form a valid triangle.");
}
```

Retest Result

Pass

- The system correctly identifies invalid inputs.
- The appropriate error message is displayed.
- No invalid shape is added.
- Valid inputs continue to produce correct results.

PART 2 - OBSERVATION

Below are the observation components that the learner is required to demonstrate as part of this assessment. The learner only needs to demonstrate this once. If the learner is unable to demonstrate one or more of the required Observation tasks to a satisfactory level, they will be given two opportunities to demonstrate the unsatisfactory Observation task(s).

Marking criteria: Skills to be observed during this task to the required standard

In this assessment task you will be performing following activities	Date 1		Date 2		Date 3		Comment
	Resubmission if applicable		Resubmission if applicable		Resubmission if applicable		
	Satisfactory		Satisfactory		Satisfactory		
	Yes	No	Yes	No	Yes	No	
Test 2D-Shape Calculator							
2.1. Determined the unit-testing needs including test framework as per the organisational standards for every class and presented all identified items as bullet points.	☐	☐	☐	☐	☐	☐	
2.2. Outlined the class-level unit-test criteria and displayed each requirement in bullet-point format.	☐	☐	☐	☐	☐	☐	
2.3. Reviewed and validated these development expectations with a member of the software development team.	☐	☐	☐	☐	☐	☐	

In this assessment task you will be performing following activities	Date 1		Date 2		Date 3		Comment
			Resubmission if applicable		Resubmission if applicable		
	Satisfactory		Satisfactory		Satisfactory		
	Yes	No	Yes	No	Yes	No	
2.4. Located the organisation’s testing standards and frameworks and listed them in bullet points.	☐	☐	☐	☐	☐	☐	
2.5. Examined at least three test-case design methods and described how each was utilised in creating the test cases.	☐	☐	☐	☐	☐	☐	
2.6. Created the test plan and corresponding test cases using the supplied template.	☐	☐	☐	☐	☐	☐	
2.7. Produced unit-test procedures through an automated testing framework.	☐	☐	☐	☐	☐	☐	
2.8. Executed manual testing following the system test plan and captured screenshots of the outcomes.	☐	☐	☐	☐	☐	☐	
2.9. Develop unit test procedures using an auto testing framework as a part of debugging. Ran automated tests in line with the unit-test plan and 2.1, recorded screenshots of the results.	☐	☐	☐	☐	☐	☐	
2.10. Discussed and confirmed the testing outcomes with a member of the software development team.	☐	☐	☐	☐	☐	☐	
2.11. Generated and stored the test reports using the provided template.	☐	☐	☐	☐	☐	☐	
2.12. Handled defects and re-evaluated the software, correcting additional issues as required.	☐	☐	☐	☐	☐	☐	
2.13. Learner identified test and script requirements including data structures	☐	☐	☐	☐	☐	☐	
2.14. Learner analysed and identified test data using multiple test-case design techniques.	☐	☐	☐	☐	☐	☐	
2.15. Learner defined and designed test cases.	☐	☐	☐	☐	☐	☐	
2.16. Learner documented test plan and script according to organisational guidelines and industry standards	☐	☐	☐	☐	☐	☐	
2.17. Learner designed and implemented algorithm required in test procedures	☐	☐	☐	☐	☐	☐	
2.18. Learner took responsibility for planning, sequencing and prioritising tasks and own workload	☐	☐	☐	☐	☐	☐	

Please use the check list below to check if you have got everything ready before submitting the assessment for review.

Part 2 - Submission check list

Assessment document with all questions answered	
All source codes	
Test plan document for system test	
Test plan document for unit test	
Test report document for system test	
Test report document for unit test	

Part 3 – Develop “ToDo” List

Scenario: To-Do List Application

You are developing a simple **To-Do List desktop application** for a small business team. The application allows users to add, edit, complete, and delete tasks. All task data will be stored locally using a single database table.

The business requires:

- Each task to have a title, description, due date, and completion status ("Not Started", "In Progress", "Completed").
- The data to be stored securely and efficiently in a database.
- The application to follow a layered architecture for future scalability.

3.1. Identify data and application requirements according to organisational procedures.

What data does your To-Do List app need to store? (e.g. title, description, due date, status)

Answer:

The To-Do list application must store:

- Title
- Description
- Due date
- Completion status ("Not Started", "In Progress", "Completed")

What features or tasks should the app allow the user to perform?

Answer:

The application should allow the user to:

- Add tasks
- Edit tasks
- Complete tasks
- Delete tasks

Are there any rules or organisational requirements you're following (e.g. naming conventions, use of local database, file storage format)?

Answer:

- The task data must be stored locally
- The task data must use a single database table
- The task data must be stored securely
- The application should follow a layered structure for future scalability

3.2 Review the function of the required DAL in a multi-layer application model which embeds the features such as constructor, polymorphism, use of appropriate access specifiers.

What is the **Data Access Layer (DAL)** in your app, and what does it do?

Answer:

The Data Access Layer is the part of the application responsible for communicating with the database. Its purpose is to separate all database operations from the rest of the application, so that other parts of the program don't need to know how the data is stored or retrieved.

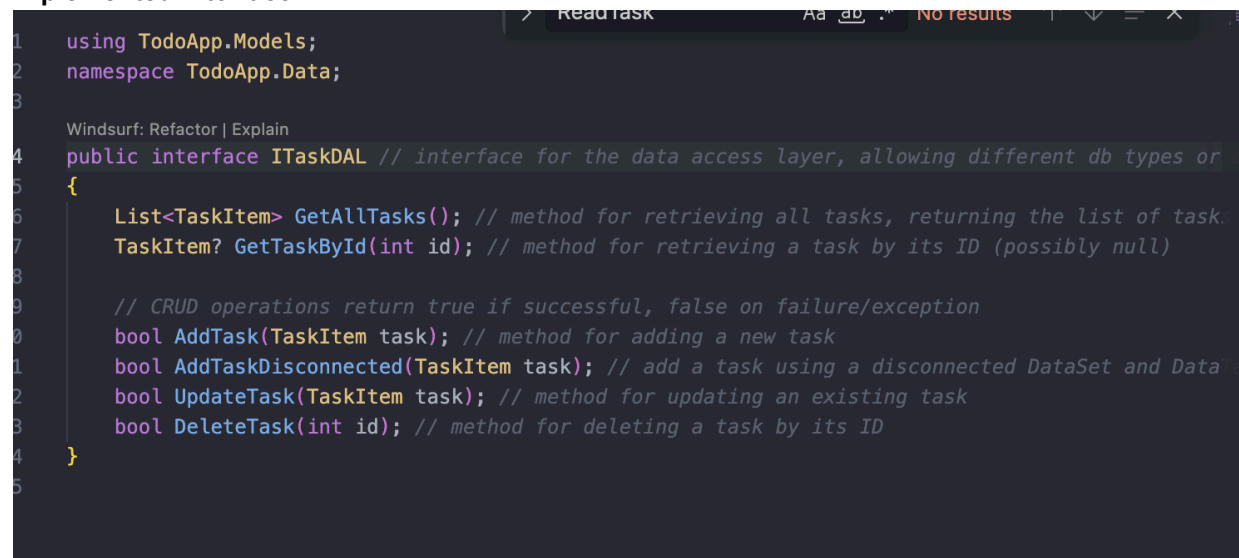
In the To-Do List application, the DAL is implemented as a SQLiteTaskDAL class. This class contains methods for connecting to the SQLite database and performing CRUD (*Create, Read, Update and Delete*) operations on the Tasks table.

The SQLiteTaskDAL uses appropriate access specifiers, such as public for methods that other parts of the application need to call and private for internal variables like the connection string. To demonstrate polymorphism, my DAL implements the interface ITaskDAL, allowing different database implementations to be used without changing the rest of the application.

Take a clear screenshot of the part of your code that handles **data access** (DAL) in your application.

Answer:

Implemented Interface:

A screenshot of a code editor showing the implementation of the ITaskDAL interface. The code is in C# and includes using statements for 'TodoApp.Models' and 'namespace TodoApp.Data;'. The interface is defined as 'public interface ITaskDAL' with several methods: 'GetAllTasks()' returning 'List<TaskItem>', 'GetTaskById(int id)' returning 'TaskItem?', and three CRUD methods: 'AddTask(TaskItem task)', 'AddTaskDisconnected(TaskItem task)', 'UpdateTask(TaskItem task)', and 'DeleteTask(int id)', all returning 'bool'. Comments explain the purpose of each method. The code is displayed in a dark-themed editor with line numbers on the left.

```
1 using TodoApp.Models;
2 namespace TodoApp.Data;
3
4 public interface ITaskDAL // interface for the data access layer, allowing different db types or
5 {
6     List<TaskItem> GetAllTasks(); // method for retrieving all tasks, returning the list of task.
7     TaskItem? GetTaskById(int id); // method for retrieving a task by its ID (possibly null)
8
9     // CRUD operations return true if successful, false on failure/exception
10    bool AddTask(TaskItem task); // method for adding a new task
11    bool AddTaskDisconnected(TaskItem task); // add a task using a disconnected DataSet and DataAdapter
12    bool UpdateTask(TaskItem task); // method for updating an existing task
13    bool DeleteTask(int id); // method for deleting a task by its ID
14 }
15
```

SQLiteTaskDAL:

```
1  using Microsoft.Data.Sqlite;
2  using TodoApp.Models;
3
4  namespace TodoApp.Data;
5
6  Windsurf: Refactor | Explain
7  public class SQLiteTaskDAL : ITaskDAL // implement the TaskDAL interface
8  {
9      private readonly string _connectionString;
10
11      Windsurf: Refactor | Explain | Generate Documentation | X
12      public SQLiteTaskDAL(string databasePath)
13      {
14          _connectionString = $"Data Source={databasePath}"; // use path to form connection string
15          CreateTable(); // create table on startup if not exists
16
17      Windsurf: Refactor | Explain | Generate Documentation | X
18      private SqliteConnection OpenConnection()
19      {
20          try // to use connection string to open a connection, return it on success
21          {
22              var connection = new SqliteConnection(_connectionString);
23              connection.Open();
24              return connection;
25          }
26          catch (SqliteException ex) // program will terminate if no connection
27          {
28              throw new InvalidOperationException("Could not connect to the task database.", ex);
29          }
30      }
31
32      Windsurf: Refactor | Explain | Generate Documentation | X
33      private void CreateTable()
34      {
35      }
```

```

30     private void CreateTable()
31     {
32         try // create the Tasks table if it does not exist
33         {
34             // using var means C# automatically disposes the connection and command when out of scope
35             using var connection = OpenConnection();
36             using var command = connection.CreateCommand();
37             command.CommandText = """
38                 CREATE TABLE IF NOT EXISTS Tasks (
39                     Id INTEGER PRIMARY KEY AUTOINCREMENT,
40                     Title TEXT NOT NULL,
41                     Description TEXT NOT NULL,
42                     DueDate TEXT NOT NULL,
43                     Status TEXT NOT NULL CHECK (Status IN ('Not Started', 'In Progress', 'Completed'))
44                 );
45             """;
46             command.ExecuteNonQuery();
47         }
48         catch (Exception ex) // on a failure app will terminate as it will be unusable w/o db creation
49         {
50             throw new InvalidOperationException(
51                 "Could not create the Tasks table.", ex);
52         }
53     }
54

```

Windsurf: Refactor | Explain | Generate Documentation | X

```

55     public List<TaskItem> GetAllTasks()
56     {
57         var tasks = new List<TaskItem>();
58
59         try // to open a connection and retrieve all tasks
60         {
61             using var connection = OpenConnection();
62             using var command = connection.CreateCommand();
63             command.CommandText = "SELECT * FROM Tasks";

```

```

64             using var reader = command.ExecuteReader();
65
66             while (reader.Read()) // while there are more tasks, read them & add to list
67                 tasks.Add(ReadTask(reader));
68         }
69         catch (Exception ex) // on failure, show the error and return an empty list
70         {
71             Console.WriteLine($"Could not retrieve tasks: {ex.Message}");
72         }
73
74         return tasks;
75     }
76

```

Windsurf: Refactor | Explain | Generate Documentation | X

```

77     public TaskItem? GetTaskById(int id)
78     {
79         try // open a connection and use the id to grab one task
80         {
81             using var connection = OpenConnection();
82             using var command = connection.CreateCommand();
83             command.CommandText = "SELECT * FROM Tasks WHERE Id = @id"; // match a task's id with the provided id
84             command.Parameters.AddWithValue("@id", id); // add the id parameter to the command
85             using var reader = command.ExecuteReader();
86
87             if (reader.Read()) // on success, return the task
88             {
89                 return ReadTask(reader);
90             }
91
92             return null; // otherwise, no task is found & return null
93         }
94         catch (Exception ex) // error handling - such as if an invalid id is provided
95         {

```

```

94     catch (Exception ex) // error handling - such as if an invalid id is provided
95     {
96         Console.WriteLine($"Could not retrieve task: {ex.Message}");
97         return null;
98     }
99 }
100

```

Windsurf: Refactor | Explain | Generate Documentation | X

```

101 public bool AddTask(TaskItem task)
102 {
103     try // open a connection and add a task
104     {
105         using var connection = OpenConnection();
106         using var command = connection.CreateCommand();
107         command.CommandText = """
108             INSERT INTO Tasks (Title, Description, DueDate, Status)
109             VALUES (@title, @description, @dueDate, @status)
110             """;
111         AddTaskParameters(command, task);
112         return command.ExecuteNonQuery() == 1;
113     }
114     catch (Exception ex)
115     {
116         Console.WriteLine($"Could not add task: {ex.Message}");
117         return false;
118     }
119 }
120

```

Windsurf: Refactor | Explain | Generate Documentation | X

```

121 public bool UpdateTask(TaskItem task)
122 {
123     try // open connection and update a task using the id
124     {

```

```

124     {
125         using var connection = OpenConnection();
126         using var command = connection.CreateCommand();
127         command.CommandText = """
128             UPDATE Tasks
129             SET Title = @title, Description = @description,
130                 DueDate = @dueDate, Status = @status
131             WHERE Id = @id
132             """;
133         AddTaskParameters(command, task);
134         command.Parameters.AddWithValue("@id", task.Id); // add id to params seperately as AddTaskParameters does
135         return command.ExecuteNonQuery() == 1; // true if 1 row effects (update successful) and false if 0 or >1 (
136     }
137     catch (Exception ex)
138     {
139         Console.WriteLine($"Could not update task: {ex.Message}");
140         return false;
141     }
142 }
143

```

Windsurf: Refactor | Explain | Generate Documentation | X

```

144 public bool DeleteTask(int id)
145 {
146     try // open connection and delete a task using the id
147     {
148         using var connection = OpenConnection();
149         using var command = connection.CreateCommand();
150         command.CommandText = "DELETE FROM Tasks WHERE Id = @id";
151         command.Parameters.AddWithValue("@id", id);
152         return command.ExecuteNonQuery() == 1; // true if 1 row effects (update successful) and false if 0 or >1 (
153     }
154     catch (Exception ex)
155     {
156         Console.WriteLine($"Could not delete task: {ex.Message}");

```

```

156         Console.WriteLine($"Could not delete task: {ex.Message}");
157         return false;
158     }
159 }
160
Windsurf: Refactor | Explain | Generate Documentation | X
161 private static void AddTaskParameters(SqliteCommand command, TaskItem task) // helper method to assign task fields
162 {
163     command.Parameters.AddWithValue("@title", task.Title);
164     command.Parameters.AddWithValue("@description", task.Description);
165     command.Parameters.AddWithValue("@dueDate", task.DueDate.ToString("yyyy-MM-dd"));
166     command.Parameters.AddWithValue("@status", task.Status);
167 }
168
Windsurf: Refactor | Explain | Generate Documentation | X
169 private static TaskItem ReadTask(SqliteDataReader reader) // helper method to read values from a table and store t
170
171 {
172     // read in values from the database and return a new TaskItem object w/ values
173     // GetOrdinal used to get column index by name, ensuring correct mapping even if the order of columns changes
174     TaskItem task = new TaskItem();
175
176     task.Id = reader.GetInt32(reader.GetOrdinal("Id"));
177     task.Title = reader.GetString(reader.GetOrdinal("Title"));
178     task.Description = reader.GetString(reader.GetOrdinal("Description"));
179     task.DueDate = DateTime.Parse(reader.GetString(reader.GetOrdinal("DueDate")));
180     task.Status = reader.GetString(reader.GetOrdinal("Status"));
181
182     return task;
183 }
184 }
185

```

Why is it important to separate your data-handling code from the rest of the application?

Answer:

Separating data-handling code from the rest of the application is important because it makes the program easier to maintain, test, and extend upon. In a multi-layer application, the Data Access Layer is responsible *only* for communicating with the database, while the user interface and business logic handle other tasks. As with all programming, having a modular approach like this and isolating responsibilities makes identifying bugs easier and reduces code duplication.

In the context of this application, it can also make developing the application more streamlined by abstracting away raw SQL, making the actual application easier and quicker to develop.

3.3 Determine data-access application programming interface (API) according to task requirements

Which library or API will you use to connect to and work with your database (e.g. Microsoft.Data.Sqlite)?

Answer:

My database will utilise the Microsoft.Data.Sqlite library to connect and work with my database.

How does this API help you perform basic database tasks like reading, writing, or updating data?

Answer:

This API helps to perform basic database tasks by providing the classes and methods needed. This includes functionality to establish database connections, execute SQL queries, retrieve data, and perform CRUD operations.

3.4 Create and manage connection strings according to task requirements

What is a connection string, and what is its purpose in your project?

Answer:

A connection string is a piece of text that contains the information needed for an application to connect to a database. It tells the database provider where the database is located and how to access it. If the database is hosted on a server, it will include information like the database name, username, and password.

Because the To-Do List application is quite simple and only makes use of a local database, the connection string just includes the file name of this database. This is used by the Microsoft.Data.Sqlite library to establish a connection by specifying the location of the file.

Provide a screenshot of the connection string used in your project.

Answer:

In SQLiteTaskDAL:

```
private readonly string _connectionString;

Windsurf: Refactor | Explain | Generate Documentation | X
public SQLiteTaskDAL(string databasePath)
{
    _connectionString = $"Data Source={databasePath}"; // use path to form connection string
    CreateTable(); // create table on startup if not exists
}
```

Path given in Program.cs:

```
ITaskDAL taskDAL = new SQLiteTaskDAL("tasks.db"); // make the DAL object w/ db path for connection string
```

3.5 Establish data connections using the appropriate data providers in line with organisational protocols.

Briefly explain how you connect to the database using the data provider you selected.

Answer:

My application uses the Microsoft.Data.Sqlite data provider to connect to a local SQLite database. The database connection is established by creating a SqlConnection object and passing it the connection string, which specifies the location of the database file.

When the application needs to access the database, it creates a new SqlConnection, opens the connection using the Open() method, and performs the required SQL operation. It will then automatically close the connection when it is no longer needed.

Provide a screenshot of the code where you establish the database connection.

Answer:

```
Windsurf: Refactor | Explain | Generate Documentation | X
private SqlConnection OpenConnection()
{
    try // to use connection string to open a connection, return it on success
    {
        var connection = new SqlConnection(_connectionString);
        connection.Open();
        return connection;
    }
    catch (SqliteException ex) // program will terminate if no connection
    {
        throw new InvalidOperationException("Could not connect to the task database.", ex);
    }
}
```

3.6 Handle Connection Exceptions using Code

Explain what exception handling is and how you used it to manage database connection errors in your project.

Answer:

Exception handling is the process of handling unexpected errors that may arise in a program gracefully. By handling exceptions, users do not encounter sudden crashes or system dialogue that makes the application unusable. Instead, possible causes for errors are identified during development, and are designed to trigger an exception. This is then handled by the program by “catching” the exception, where the developer can then tell the program what logic to execute, rather than having the program hang or crash when something unexpected occurs.

In my To-Do project, one area where I implemented exception handling was with establishing a connection to the database. There is a high possibility for errors here, such as if the wrong file name is provided in the connection string. Without correct exception handling, this would cause the program to crash, and the user would have no information as to what went wrong. Instead, the program throws an exception if this happens, and the application will give the user an informative error message rather than simply quitting.

Provide a screenshot of the code where you handle database connection exceptions.

Answer:

```
private SqlConnection OpenConnection()
{
    try // to use connection string to open a connection, return it on success
    {
        var connection = new SqlConnection(_connectionString);
        connection.Open();
        return connection;
    }
    catch (SqliteException ex) // program will terminate if no connection
    {
        throw new InvalidOperationException("Could not connect to the task database.", ex);
    }
}
```

3.7 Execute CRUD commands and return results from data source

Your code must be **fully commented** to explain what each part does. You are required to include these comments in your code screenshots as part of your submission.

Read All Task

Provide a screenshot of your function that reads all tasks from the database.

Your code must include:

- A SELECT * query
- Retrieving the result set (e.g. using a loop or data reader)
- A try-catch block to handle exceptions

Also include two screenshots as test evidence:

- A screenshot of the task records in your database (e.g. from DB Browser for SQLite)
- A screenshot showing the output of your function when it runs (e.g. console output or app interface)

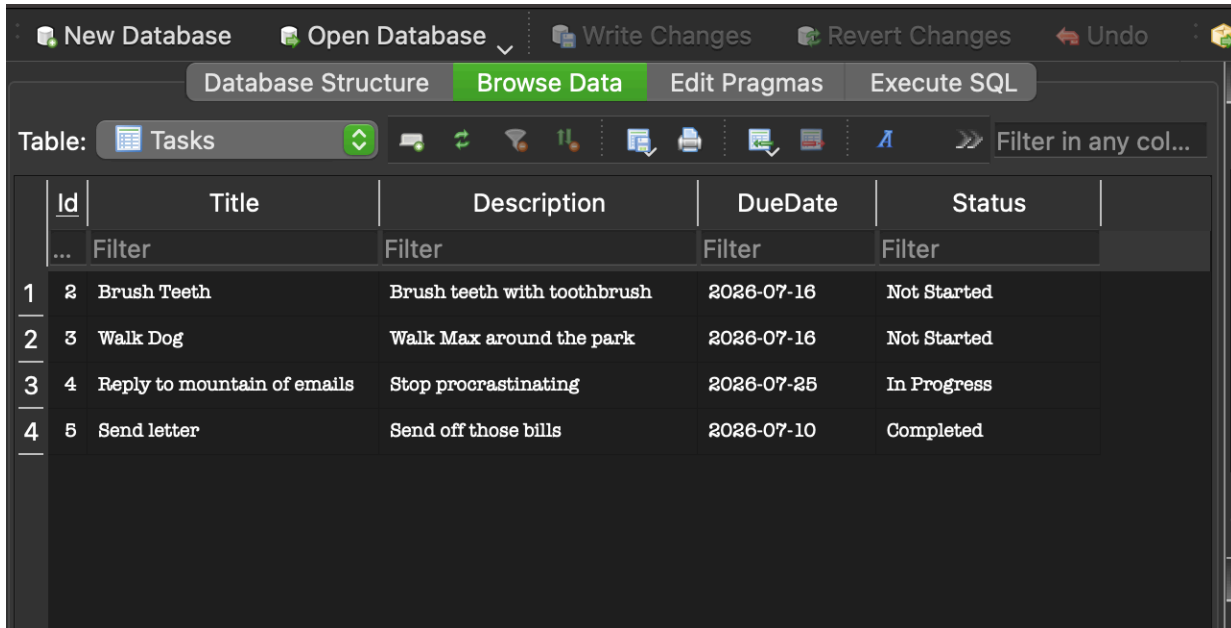
Answer:

Screenshot of your code (exception must be handled, e.g., using try-catch block)

```
public List<TaskItem> GetAllTasks()  
{  
    var tasks = new List<TaskItem>();  
  
    try // to open a connection and retrieve all tasks  
    {  
        using var connection = OpenConnection();  
        using var command = connection.CreateCommand();  
        command.CommandText = "SELECT * FROM Tasks";  
        using var reader = command.ExecuteReader();  
  
        while (reader.Read()) // while there are more tasks, read them & add to list  
        {  
            tasks.Add(ReadTask(reader));  
        }  
    }  
    catch (Exception ex) // on failure, show the error and return an empty list  
    {  
        Console.WriteLine($"Could not retrieve tasks: {ex.Message}");  
    }  
  
    return tasks;  
}
```

```
private static TaskItem ReadTask(SqliteDataReader reader) // helper method to read values from a table and store  
  
    // read in values from the database and return a new TaskItem object w/ values  
    // GetOrdinal used to get column index by name, ensuring correct mapping even if the order of columns changes  
    TaskItem task = new TaskItem();  
  
    task.Id = reader.GetInt32(reader.GetOrdinal("Id"));  
    task.Title = reader.GetString(reader.GetOrdinal("Title"));  
    task.Description = reader.GetString(reader.GetOrdinal("Description"));  
    task.DueDate = DateTime.Parse(reader.GetString(reader.GetOrdinal("DueDate")));  
    task.Status = reader.GetString(reader.GetOrdinal("Status"));  
  
    return task;
```

Screenshot of the task records in your database



The screenshot shows a database application interface. At the top, there are buttons for 'New Database', 'Open Database', 'Write Changes', 'Revert Changes', and 'Undo'. Below these are tabs for 'Database Structure', 'Browse Data' (which is active), 'Edit Pragmas', and 'Execute SQL'. The 'Table:' dropdown is set to 'Tasks'. Below the table name, there are icons for various database operations. The main area displays a table with the following data:

	<u>Id</u>	Title	Description	DueDate	Status
...	Filter		Filter	Filter	Filter
1	2	Brush Teeth	Brush teeth with toothbrush	2026-07-16	Not Started
2	3	Walk Dog	Walk Max around the park	2026-07-16	Not Started
3	4	Reply to mountain of emails	Stop procrastinating	2026-07-25	In Progress
4	5	Send letter	Send off those bills	2026-07-10	Completed

Screenshot showing the output of your function when it runs

```
1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
0. Exit
Enter Choice: 1
2: Brush Teeth | Brush teeth with toothbrush | 2026-07-16 | Not Started
3: Walk Dog | Walk Max around the park | 2026-07-16 | Not Started
4: Reply to mountain of emails | Stop procrastinating | 2026-07-25 | In Progress
5: Send letter | Send off those bills | 2026-07-10 | Completed
```

I confirm that the data displayed by my program matches the data stored in the database, based on the screenshots provided.

Read One Task

Provide a screenshot of your function that reads one task by ID from the database.

Your code must include:

- A parameterized query using the task ID
- Retrieving and returning the matching result
- A try-catch block to handle any potential errors

Also include two screenshots as test evidence:

- A screenshot of the task records in your database (e.g. from DB Browser for SQLite)

A screenshot showing the output of your function when it runs (e.g. console output or app interface)

Answer:

Screenshot of your code (exception must be handled, e.g., using try-catch block)

```
public TaskItem? GetTaskById(int id)
{
    try // open a connection and use the id to grab one task
    {
        using var connection = OpenConnection();
        using var command = connection.CreateCommand();
        command.CommandText = "SELECT * FROM Tasks WHERE Id = @id"; // match a task's id with the provided id
        command.Parameters.AddWithValue("@id", id); // add the id parameter to the command
        using var reader = command.ExecuteReader();

        if (reader.Read()) // on success, return the task
        {
            return ReadTask(reader);
        }

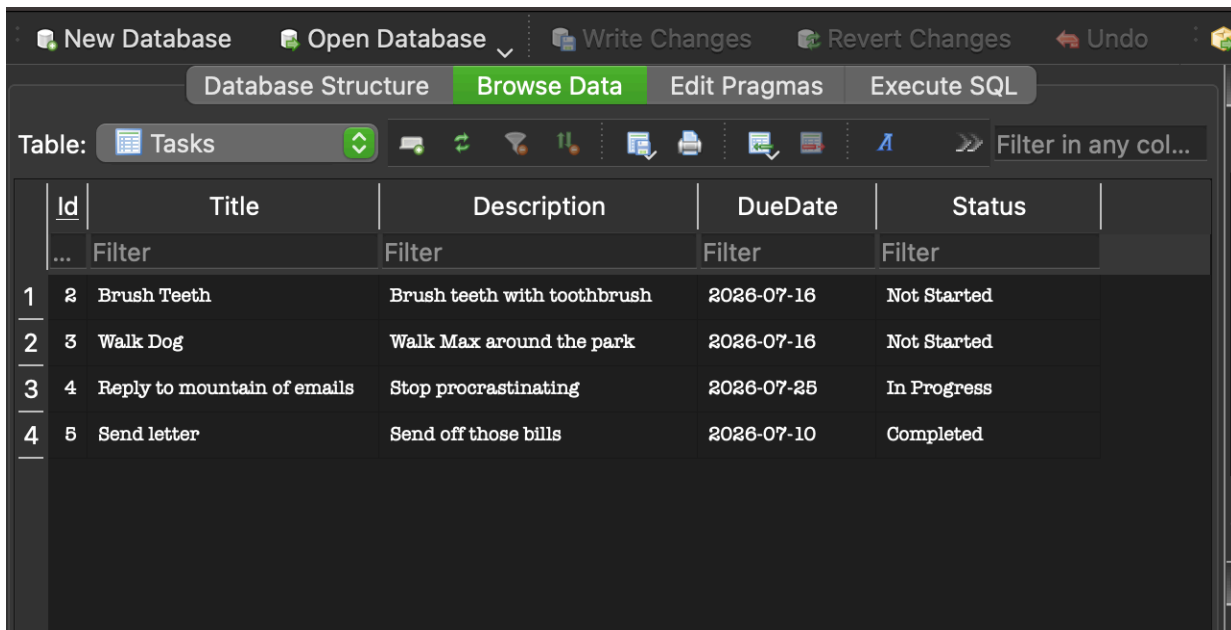
        return null; // otherwise, no task is found & return null
    }
    catch (Exception ex) // error handling - such as if an invalid id is provided
    {
        Console.WriteLine($"Could not retrieve task: {ex.Message}");
        return null;
    }
}
```

```
private static TaskItem ReadTask(SqliteDataReader reader) // helper method to read values from a table and store
{
    // read in values from the database and return a new TaskItem object w/ values
    // GetOrdinal used to get column index by name, ensuring correct mapping even if the order of columns changes
    TaskItem task = new TaskItem();

    task.Id = reader.GetInt32(reader.GetOrdinal("Id"));
    task.Title = reader.GetString(reader.GetOrdinal("Title"));
    task.Description = reader.GetString(reader.GetOrdinal("Description"));
    task.DueDate = DateTime.Parse(reader.GetString(reader.GetOrdinal("DueDate")));
    task.Status = reader.GetString(reader.GetOrdinal("Status"));

    return task;
}
```

Screenshot of the task records in your database



The screenshot shows a database application interface. At the top, there are buttons for 'New Database', 'Open Database', 'Write Changes', 'Revert Changes', and 'Undo'. Below these are tabs for 'Database Structure', 'Browse Data' (which is selected), 'Edit Pragmas', and 'Execute SQL'. Under the 'Browse Data' tab, there is a 'Table:' dropdown menu showing 'Tasks'. Below the table name, there is a toolbar with various icons. The main area displays a table with the following data:

		<u>Id</u>	Title	Description	DueDate	Status
	...	Filter		Filter	Filter	Filter
1	2	Brush Teeth	Brush teeth with toothbrush	2026-07-16	Not Started	
2	3	Walk Dog	Walk Max around the park	2026-07-16	Not Started	
3	4	Reply to mountain of emails	Stop procrastinating	2026-07-25	In Progress	
4	5	Send letter	Send off those bills	2026-07-10	Completed	

Screenshot showing the output of your function when it runs

```
1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
0. Exit
Enter Choice: 1
2: Brush Teeth | Brush teeth with toothbrush | 2026-07-16 | Not Started
3: Walk Dog | Walk Max around the park | 2026-07-16 | Not Started
4: Reply to mountain of emails | Stop procrastinating | 2026-07-25 | In Progress
5: Send letter | Send off those bills | 2026-07-10 | Completed

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
0. Exit
Enter Choice: 2
Enter Task ID: 4
4: Reply to mountain of emails | Stop procrastinating | 2026-07-25 | In Progress
```

I confirm that the data displayed by my program matches the data stored in the database, based on the screenshots provided.

Insert Task

Provide a screenshot of your function that inserts a new task into the database.

Your code must include:

- A parameterized INSERT query
- Proper use of parameters for task fields (e.g. title, description, due date, status)
- A try-catch block to handle any potential errors

Also include two screenshots as test evidence:

- A screenshot of the new task record in your database (e.g. from DB Browser for SQLite)
- A screenshot showing the output of your function after inserting the task (e.g. console message or UI confirmation)

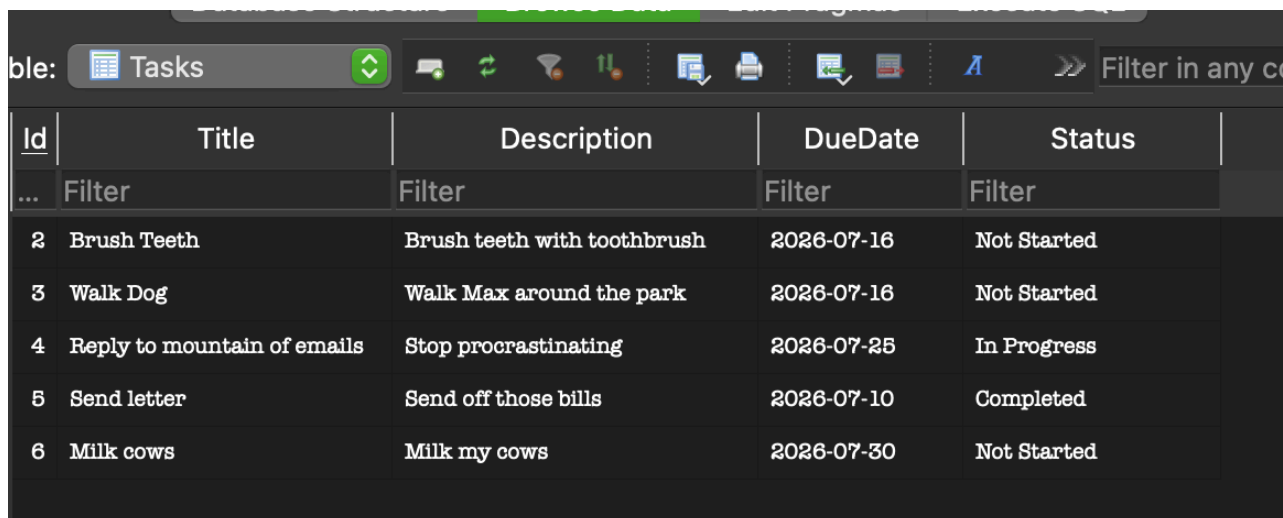
Answer:

Screenshot of your code (exception must be handled, e.g., using try-catch block)

```
public bool AddTask(TaskItem task)
{
    try // open a connection and add a task
    {
        using var connection = OpenConnection();
        using var command = connection.CreateCommand();
        command.CommandText = """
            INSERT INTO Tasks (Title, Description, DueDate, Status)
            VALUES (@title, @description, @dueDate, @status)
            """;
        AddTaskParameters(command, task);
        return command.ExecuteNonQuery() == 1;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Could not add task: {ex.Message}");
        return false;
    }
}
```

```
private static void AddTaskParameters(SqliteCommand command, TaskItem task) // helper method to assign task fields
{
    command.Parameters.AddWithValue("@title", task.Title);
    command.Parameters.AddWithValue("@description", task.Description);
    command.Parameters.AddWithValue("@dueDate", task.DueDate.ToString("yyyy-MM-dd"));
    command.Parameters.AddWithValue("@status", task.Status);
}
```

Screenshot of the task records in your database



Id	Title	Description	DueDate	Status
2	Brush Teeth	Brush teeth with toothbrush	2026-07-16	Not Started
3	Walk Dog	Walk Max around the park	2026-07-16	Not Started
4	Reply to mountain of emails	Stop procrastinating	2026-07-25	In Progress
5	Send letter	Send off those bills	2026-07-10	Completed
6	Milk cows	Milk my cows	2026-07-30	Not Started

Screenshot showing the output of your function when it runs

```
1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
0. Exit
Enter Choice: 3
Title: Milk cows
Description: Milk my cows
Due date (yyyy-MM-dd): 2026-07-30
Select a status:
1. Not Started
2. In Progress
3. Completed
Status: 1
Task added.
```

I confirm that the data displayed by my program matches the data stored in the database, based on the screenshots provided.

Update Task

Provide a screenshot of your function that updates an existing task in the database.

Your code must include:

- A parameterized UPDATE query using the task ID
- Logic to update one or more task fields (e.g. title or status)
- A try-catch block to handle any potential errors

Also include two screenshots as test evidence:

- A screenshot of the updated task record in your database
- A screenshot showing the output of your function confirming the update (e.g. "Task updated successfully")

Answer:

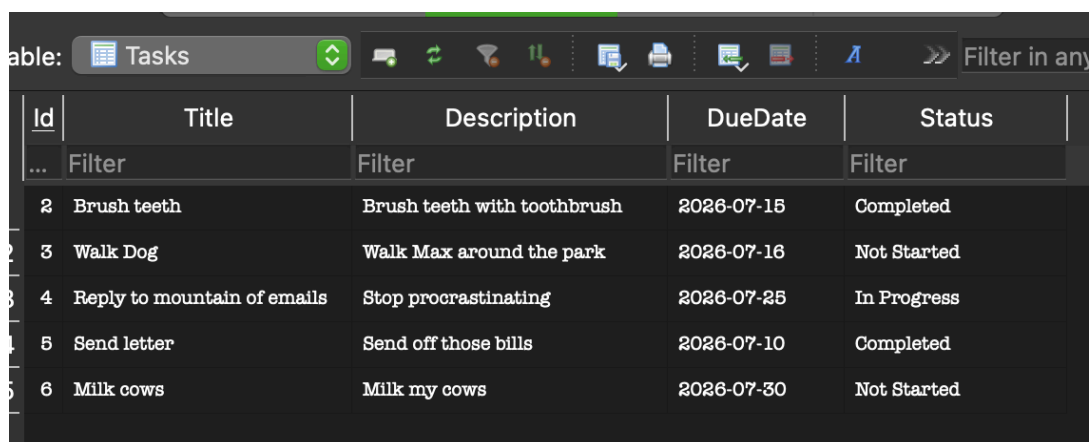
Screenshot of your code (exception must be handled, e.g., using try-catch block)

```
public bool UpdateTask(TaskItem task)
{
    try // open connection and update a task using the id
    {
        using var connection = OpenConnection();
        using var command = connection.CreateCommand();
        command.CommandText = """
            UPDATE Tasks
            SET Title = @title, Description = @description,
                DueDate = @dueDate, Status = @status
            WHERE Id = @id
            """;
        AddTaskParameters(command, task);
        command.Parameters.AddWithValue("@id", task.Id); // add id to params seperately as AddTaskParameters does not
        return command.ExecuteNonQuery() == 1; // true if 1 row effects (update successful) and false if 0 or >1 (update failed)
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Could not update task: {ex.Message}");
        return false;
    }
}

private static void AddTaskParameters(SqliteCommand command, TaskItem task) // helper method to assign task fields
{
    command.Parameters.AddWithValue("@title", task.Title);
    command.Parameters.AddWithValue("@description", task.Description);
    command.Parameters.AddWithValue("@dueDate", task.DueDate.ToString("yyyy-MM-dd"));
    command.Parameters.AddWithValue("@status", task.Status);
}
```

Screenshot of the task records in your database

(Changed Task 2 to be "Completed")



Id	Title	Description	DueDate	Status
2	Brush teeth	Brush teeth with toothbrush	2026-07-15	Completed
3	Walk Dog	Walk Max around the park	2026-07-16	Not Started
4	Reply to mountain of emails	Stop procrastinating	2026-07-25	In Progress
5	Send letter	Send off those bills	2026-07-10	Completed
6	Milk cows	Milk my cows	2026-07-30	Not Started

Screenshot showing the output of your function when it runs

(Changed Task 2 to be "Completed")

```
Enter Choice: 1
2: Brush Teeth | Brush teeth with toothbrush | 2026-07-16 | Not Started
3: Walk Dog | Walk Max around the park | 2026-07-16 | Not Started
4: Reply to mountain of emails | Stop procrastinating | 2026-07-25 | In Progress
5: Send letter | Send off those bills | 2026-07-10 | Completed
6: Milk cows | Milk my cows | 2026-07-30 | Not Started

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
0. Exit
Enter Choice: 4
Enter Task ID: 2
Title: Brush teeth
Description: Brush teeth with toothbrush
Due date (yyyy-MM-dd): 2026-07-15
Select a status:
1. Not Started
2. In Progress
3. Completed
Status: 3
Task updated.

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
0. Exit
Enter Choice: 1
2: Brush teeth | Brush teeth with toothbrush | 2026-07-15 | Completed
3: Walk Dog | Walk Max around the park | 2026-07-16 | Not Started
4: Reply to mountain of emails | Stop procrastinating | 2026-07-25 | In Progress
5: Send letter | Send off those bills | 2026-07-10 | Completed
6: Milk cows | Milk my cows | 2026-07-30 | Not Started

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
0. Exit
Enter Choice: █
```

I confirm that the data displayed by my program matches the data stored in the database, based on the screenshots provided.

Delete Task

Provide a screenshot of your function that deletes a task from the database using its ID.

Your code must include:

- A parameterized DELETE query using the task ID
- Logic to confirm whether the delete was successful
- A try-catch block to handle any potential errors

Also include two screenshots as test evidence:

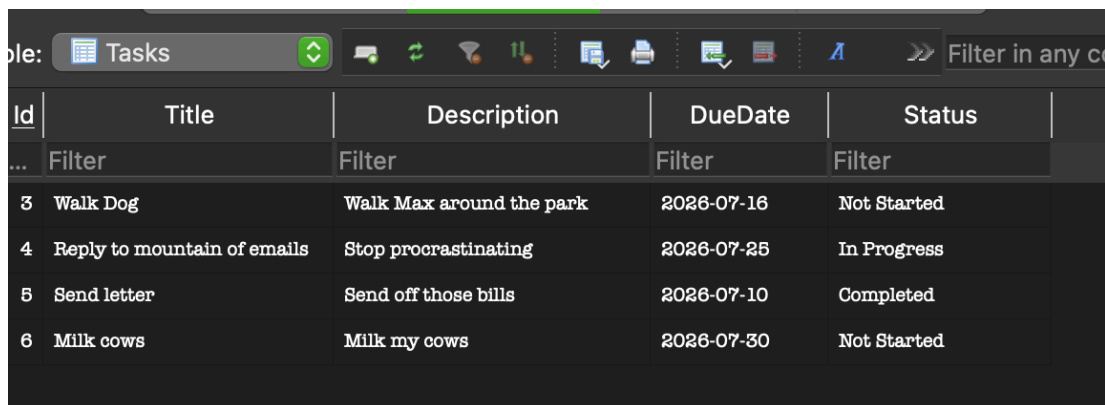
- A screenshot of the database before and after the delete to show the change
- A screenshot of your program's output confirming the task was deleted

Answer:

Screenshot of your code (exception must be handled, e.g., using try-catch block)

```
public bool DeleteTask(int id)
{
    try // open connection and delete a task using the id
    {
        using var connection = OpenConnection();
        using var command = connection.CreateCommand();
        command.CommandText = "DELETE FROM Tasks WHERE Id = @id";
        command.Parameters.AddWithValue("@id", id);
        return command.ExecuteNonQuery() == 1; // true if 1 row effects (delete successful)
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Could not delete task: {ex.Message}");
        return false;
    }
}
```

Screenshot of the task records in your database



Id	Title	Description	DueDate	Status
3	Walk Dog	Walk Max around the park	2026-07-16	Not Started
4	Reply to mountain of emails	Stop procrastinating	2026-07-25	In Progress
5	Send letter	Send off those bills	2026-07-10	Completed
6	Milk cows	Milk my cows	2026-07-30	Not Started

Screenshot showing the output of your function when it runs

```

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
0. Exit
Enter Choice: 1
2: Brush teeth | Brush teeth with toothbrush | 2026-07-15 | Completed
3: Walk Dog | Walk Max around the park | 2026-07-16 | Not Started
4: Reply to mountain of emails | Stop procrastinating | 2026-07-25 | In Progress
5: Send letter | Send off those bills | 2026-07-10 | Completed
6: Milk cows | Milk my cows | 2026-07-30 | Not Started

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
0. Exit
Enter Choice: 5
Enter Task ID: 2
Task deleted.

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
0. Exit
Enter Choice: 1
3: Walk Dog | Walk Max around the park | 2026-07-16 | Not Started
4: Reply to mountain of emails | Stop procrastinating | 2026-07-25 | In Progress
5: Send letter | Send off those bills | 2026-07-10 | Completed
6: Milk cows | Milk my cows | 2026-07-30 | Not Started

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
0. Exit
Enter Choice: █

```

I confirm that the data displayed by my program matches the data stored in the database, based on the screenshots provided.

PART 3 - OBSERVATION

Below are the observation components that the learner is required to demonstrate as part of this assessment. The learner only needs to demonstrate2 this once. If the learner is unable to demonstrate one or more of the required Observation tasks to a satisfactory level, they will be given two opportunities to demonstrate the unsatisfactory Observation task(s).

Marking criteria: Skills to be observed during this task to the required standard

In this assessment task you will be performing following activities	Date 1		Date 2		Date 3		Comment
	Resubmission if applicable		Resubmission if applicable		Resubmission if applicable		
	Satisfactory Yes No		Satisfactory Yes No		Satisfactory Yes No		
Develop “ToDo” List							
3.7.1. Learner queried data from data source according to task requirements	☐	☐	☐	☐	☐	☐	
3.7.2. Learner applied documentation conventions and documented the application according to organisational documentation conventions	☐	☐	☐	☐	☐	☐	
3.7.3. Learner retrieved and managed results set from data source	☐	☐	☐	☐	☐	☐	

In this assessment task you will be performing following activities	Date 1		Date 2		Date 3		Comment
	Satisfactory		Resubmission if applicable Satisfactory		Resubmission if applicable Satisfactory		
	Yes	No	Yes	No	Yes	No	
3.7.4. Learner managed exceptions when retrieving data	☐	☐	☐	☐	☐	☐	
3.7.5. Learner document code and database connectivity according to organisational procedures	☐	☐	☐	☐	☐	☐	

I confirm that I have submitted the documented code.

3.8 Research disconnected data management strategy

What is disconnected mode in data access, and what are its benefits and limitations?

In your own words, explain:

- What "disconnected mode" means
- How it differs from connected mode
- One or two advantages (e.g. better performance, less load on the database)
- One or two challenges (e.g. managing updates or conflicts)

Answer:

Disconnected mode is a method of accessing database data where the application retrieves the required data from the database, stores it in memory, and then closes the database connection; The application works with the data in memory instead of keeping the connection to the database open. This differs from connected mode, where the application maintains the connection to the database while reading or updating data.

Some advantages of disconnected mode include that the connection is only open for a short time, reducing the load on the database server and possibly improving performance when working with the local data, if the same data needs to be accessed multiple times. It also allows users to continue working with data, even after the database connection has been closed.

However, some challenges of disconnected mode include that, importantly, changes made to the data in memory are not automatically saved to the database and must be updated *manually*. This means if multiple users modify the same data at the same time, conflicts can occur when the changes are written back to the database. Also, storing large amounts of data in memory may increase the application's memory usage if the dataset is large.

Implement at least one CRUD operation (Insert, Update, or Delete) using disconnected mode, and provide evidence.

Your code must use a disconnected data access approach (e.g. DataSet, DataTable, and DataAdapter) and include proper exception handling.

Provide three screenshots:

- Your code that performs the CRUD operation in disconnected mode, with comments
- A screenshot of the relevant data in your database (e.g. from DB Browser for SQLite)
- A screenshot showing your program's execution result (e.g. console output or confirmation in the interface)

Answer:

Screenshot of your code (exception must be handled, e.g., using try-catch block)

Within SQLiteTaskDAL:

```
public bool AddTaskDisconnected(TaskItem task) // disconnected mode for adding a task
{
    try // to store the new task in memory without an open database connection
    {
        // temp. store data in memory using a DataSet and DataTable
        using DataSet taskDataSet = new DataSet();
        DataTable taskTable = new DataTable("Tasks");

        taskTable.Columns.Add("Title", typeof(string));
        taskTable.Columns.Add("Description", typeof(string));
        taskTable.Columns.Add("DueDate", typeof(string));
        taskTable.Columns.Add("Status", typeof(string));
        taskDataSet.Tables.Add(taskTable);

        DataRow newRow = taskTable.NewRow();
        newRow["Title"] = task.Title;
        newRow["Description"] = task.Description;
        newRow["DueDate"] = task.DueDate.ToString("yyyy-MM-dd");
        newRow["Status"] = task.Status;
        taskTable.Rows.Add(newRow);

        // reconnect once the disconnected data is ready to be saved
        // in practice this could be done in a batch for multiple tasks
        // the connection will be closed control leaves this block
        using var connection = OpenConnection();
        using var command = connection.CreateCommand();
        command.CommandText = """
            INSERT INTO Tasks (Title, Description, DueDate, Status)
            VALUES (@title, @description, @dueDate, @status)
            """;

        // cannot use helper method AddTaskParameters here as it takes a TaskItem object, not a DataRow
        command.Parameters.AddWithValue("@title", newRow["Title"]);
        command.Parameters.AddWithValue("@description", newRow["Description"]);
        command.Parameters.AddWithValue("@dueDate", newRow["DueDate"]);
        command.Parameters.AddWithValue("@status", newRow["Status"]);

        return command.ExecuteNonQuery() == 1; // true if 1 row effects (insert successful) and false if 0 or >1 (ins
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Could not add task in disconnected mode: {ex.Message}");
        return false;
    }
}
```

Screenshot of the task records in your database

ole: Tasks				
Id	Title	Description	DueDate	Status
...	Filter	Filter	Filter	Filter
3	Walk Dog	Walk Max around the park	2026-07-16	Not Started
4	Reply to mountain of emails	Stop procrastinating	2026-07-25	In Progress
5	Send letter	Send off those bills	2026-07-10	Completed
6	Milk cows	Milk my cows	2026-07-30	Not Started
7	Rob bank	If i add this in disconnected ...	2026-07-20	Not Started

Screenshot showing the output of your function when it runs

```
4. Update Task
5. Delete Task
6. Add Task (Disconnected Mode)
0. Exit
Enter Choice: 1
3: Walk Dog | Walk Max around the park | 2026-07-16 | Not Started
4: Reply to mountain of emails | Stop procrastinating | 2026-07-25 | In Progress
5: Send letter | Send off those bills | 2026-07-10 | Completed
6: Milk cows | Milk my cows | 2026-07-30 | Not Started

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
6. Add Task (Disconnected Mode)
0. Exit
Enter Choice: 6
Title: Rob bank
Description: If i add this in disconnected mode, no one will know!
Due date (yyyy-MM-dd): 2026-07-20
Select a status:
1. Not Started
2. In Progress
3. Completed
Status: 1
Task added using disconnected mode.

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
6. Add Task (Disconnected Mode)
0. Exit
Enter Choice: 1
3: Walk Dog | Walk Max around the park | 2026-07-16 | Not Started
4: Reply to mountain of emails | Stop procrastinating | 2026-07-25 | In Progress
5: Send letter | Send off those bills | 2026-07-10 | Completed
6: Milk cows | Milk my cows | 2026-07-30 | Not Started
7: Rob bank | If i add this in disconnected mode, no one will know! | 2026-07-20 | Not Started

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
6. Add Task (Disconnected Mode)
0. Exit
Enter Choice: █
```

I confirm that the data displayed by my program matches the data stored in the database, based on the screenshots provided.

3.9 CSV File Import and Export

Write a function that reads task records from a **CSV file** and inserts them into your database.

Your code must:

- Read data from a properly formatted CSV file
- Parse each row and insert it into the database
- Handle exceptions

Provide three screenshots:

- Your import function code with comments
- The CSV file content (open in a text editor or spreadsheet)
- The database view after import (e.g. in DB Browser or a SELECT output from your program)

Answer:

Screenshot of your code

SQLiteTaskDAL:

```
171     public int ImportTasksFromCsv(string filePath) // takes a file path to a CSV file and imports the tasks into the
172     {
173         int tasksInserted = 0;
174
175         try // to parse the CSV file and insert tasks into the database
176         {
177             using TextFieldParser parser = new TextFieldParser(filePath);
178             parser.TextFieldType = FieldType.Delimited; // will be delimited by commas
179             parser.SetDelimiters(","); // comma between values
180             parser.HasFieldsEnclosedInQuotes = true; // allows for fields to be enclosed in quotes
181
182             string[]? headings = parser.ReadFields(); // read the headings
183
184             // validate the format of the headings
185             // must not be null, must be correct length, must match the expected headings (case-insensitive)
186             if (headings == null || headings.Length != 4 ||
187                 !headings[0].Equals("Title", StringComparison.OrdinalIgnoreCase) ||
188                 !headings[1].Equals("Description", StringComparison.OrdinalIgnoreCase) ||
189                 !headings[2].Equals("DueDate", StringComparison.OrdinalIgnoreCase) ||
190                 !headings[3].Equals("Status", StringComparison.OrdinalIgnoreCase))
191             {
192                 throw new FormatException(
193                     "CSV headings must be: Title,Description,DueDate,Status");
194             }
195
196             while (!parser.EndOfData) // there's still data to be read
197             {
198                 string[]? fields = parser.ReadFields();
199             }
```

```

200 // check for invalid row format
201 if (fields == null || fields.Length != 4)
202 {
203     throw new FormatException( $"CSV row {parser.LineNumber} must contain four fields.");
204 }
205
206 // validate and parse the date
207 bool validDate = DateTime.TryParseExact(
208     fields[2],
209     "yyyy-MM-dd",
210     CultureInfo.InvariantCulture,
211     DateTimeStyles.None,
212     out DateTime dueDate);
213
214 if (!validDate)
215 {
216     throw new FormatException(
217         $"CSV row {parser.LineNumber} has an invalid date. Use yyyy-MM-dd.");
218 }
219
220 // check for valid status
221 if (fields[3] != "Not Started" &&
222     fields[3] != "In Progress" &&
223     fields[3] != "Completed")
224 {
225     throw new FormatException(
226         $"CSV row {parser.LineNumber} has an invalid status.");
227 }

```

```

227 }
228
229 // all validation passed, assign values
230 TaskItem task = new TaskItem();
231 task.Title = fields[0];
232 task.Description = fields[1];
233 task.DueDate = dueDate; // date parsed and assigned in validation efforts
234 task.Status = fields[3];
235
236 if (!AddTask(task)) // cant add the new task for some reason
237 {
238     throw new InvalidOperationException(
239         $"The task on CSV row {parser.LineNumber} could not be inserted.");
240 }
241
242 tasksInserted++; // move to the next task
243 }
244
245 return tasksInserted; // num. tasks inserted
246 }
247 catch (Exception ex)
248 {
249     Console.WriteLine($"Could not import tasks from CSV: {ex.Message}");
250     return -1; // obviously failed (can't insert -1 tasks!)
251 }
252 }
253

```

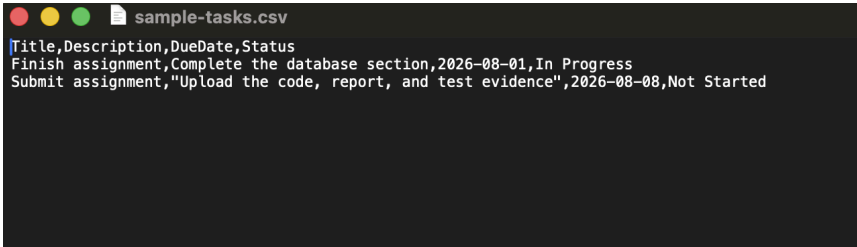
Windsurf: Refactor | Explain | Generate Documentation | X

Program.cs:

```
case "7": // import tasks from a CSV file
    Console.WriteLine("CSV file path: ");
    string csvFilePath = Console.ReadLine() ?? "";
    int importedCount = taskDAL.ImportTasksFromCsv(csvFilePath);

    if (importedCount >= 0) // tell the user how many tasks were successfully inserted
    {
        Console.WriteLine($"{importedCount} task(s) imported.");
    }
    else
    {
        Console.WriteLine("CSV import failed.");
    }
    break;
```

Screenshot of the CSV file content



```
sample-tasks.csv
Title,Description,DueDate,Status
Finish assignment,Complete the database section,2026-08-01,In Progress
Submit assignment,"Upload the code, report, and test evidence",2026-08-08,Not Started
```

Screenshot database view after import



```
1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
6. Add Task (Disconnected Mode)
7. Import Tasks from CSV
0. Exit
Enter Choice: 7
CSV file path: TodoApp/sample-tasks.csv
2 task(s) imported.

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
6. Add Task (Disconnected Mode)
7. Import Tasks from CSV
0. Exit
Enter Choice: 1
3: Walk Dog | Walk Max around the park | 2026-07-16 | Not Started
4: Reply to mountain of emails | Stop procrastinating | 2026-07-25 | In Progress
5: Send letter | Send off those bills | 2026-07-10 | Completed
6: Milk cows | Milk my cows | 2026-07-30 | Not Started
7: Rob bank | If i add this in disconnected mode, no one will know! | 2026-07-20 | Not Started
8: Finish assignment | Complete the database section | 2026-08-01 | In Progress
9: Submit assignment | Upload the code, report, and test evidence | 2026-08-08 | Not Started

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
6. Add Task (Disconnected Mode)
7. Import Tasks from CSV
0. Exit
Enter Choice: █
```

Table: Tasks Filter in any col.

	<u>Id</u>	Title	Description	DueDate	Status
...	Filter	Filter	Filter	Filter	
1	3	Walk Dog	Walk Max around the park	2026-07-16	Not Started
2	4	Reply to mountain of emails	Stop procrastinating	2026-07-25	In Progress
3	5	Send letter	Send off those bills	2026-07-10	Completed
4	6	Milk cows	Milk my cows	2026-07-30	Not Started
5	7	Rob bank	If i add this in disconnected ...	2026-07-20	Not Started
6	8	Finish assignment	Complete the database section	2026-08-01	In Progress
7	9	Submit assignment	Upload the code, report, and ...	2026-08-08	Not Started

I confirm that the tasks from the CSV file were successfully added to the database.

Write a function that queries all task records from your database and writes them into a CSV file.

Your code must:

- Execute a `SELECT * FROM Tasks` query
- Write the results to a new CSV file, including headers
- Handle exceptions

Provide three screenshots:

- Your export function code with comments
- The database view of the data to export (e.g. in DB Browser or a `SELECT` output from your program)
- The CSV file content after export (open in a text editor or spreadsheet)

Answer:

Screenshot of your code (exception must be handled, e.g., using try-catch block)

Program.cs:

```
case "8": // export all tasks to a new CSV file
    Console.WriteLine("New CSV file path: ");
    string exportFilePath = Console.ReadLine() ?? "";
    bool exportSucceeded = taskDAL.ExportTasksToCsv(exportFilePath);

    if (exportSucceeded)
    {
        Console.WriteLine("Tasks exported successfully.");
    }
    else
    {
        Console.WriteLine("CSV export failed.");
    }
    break;
```

SQLiteTaskDAL:

```
public bool ExportTasksToCsv(string filePath)
{
    try // to open a connection and save the db content to a csv
    {
        using var connection = OpenConnection();
        using var command = connection.CreateCommand();
        command.CommandText = "SELECT * FROM Tasks";
        using var reader = command.ExecuteReader();
        using StreamWriter writer = new StreamWriter(filePath, false);

        writer.WriteLine("Id,Title,Description,DueDate,Status"); // add the header

        while (reader.Read()) // there's more to read
        {
            // assign variables
            string id = reader["Id"].ToString() ?? "";
            string title = EscapeCsv(reader["Title"].ToString() ?? "");
            string description = EscapeCsv(reader["Description"].ToString() ?? "");
            string dueDate = EscapeCsv(reader["DueDate"].ToString() ?? "");
            string status = EscapeCsv(reader["Status"].ToString() ?? "");

            // write to file in the correct format
            writer.WriteLine($"{id},{title},{description},{dueDate},{status}");
        }

        return true; // success!
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Could not export tasks to CSV: {ex.Message}");
        return false; // boo failure
    }
}
```

```
private static string EscapeCsv(string value) // helper used to check individual values for special characters
{
    if (value.Contains(',') || value.Contains('"') ||
        value.Contains('\n') || value.Contains('\r'))
    {
        // escaping spec. chars prevents commas/quotes/line breaks corrupting the csv column structure
        return "\"" + value.Replace("\"", "\\\"") + "\"";
    }

    return value; // return escaped string
}
```

Screenshot database view for export

```

3. Add Task
4. Update Task
5. Delete Task
6. Add Task (Disconnected Mode)
7. Import Tasks from CSV
8. Export Tasks to CSV
0. Exit
Enter Choice: 1
3: Walk Dog | Walk Max around the park | 2026-07-16 | Not Started
4: Reply to mountain of emails | Stop procrastinating | 2026-07-25 | In Progress
5: Send letter | Send off those bills | 2026-07-10 | Completed
6: Milk cows | Milk my cows | 2026-07-30 | Not Started
7: Rob bank | If i add this in disconnected mode, no one will know! | 2026-07-20 | Not Started
8: Finish assignment | Complete the database section | 2026-08-01 | In Progress
9: Submit assignment | Upload the code, report, and test evidence | 2026-08-08 | Not Started

1. List All Tasks
2. View One Task
3. Add Task
4. Update Task
5. Delete Task
6. Add Task (Disconnected Mode)
7. Import Tasks from CSV
8. Export Tasks to CSV
0. Exit
Enter Choice: 8
New CSV file path: exported-tasks.csv
Tasks exported successfully.

```

Database Structure Browse Data Edit Pragmas Execute SQL					
Table:	Tasks				Filter in any c
	<u>Id</u>	Title	Description	DueDate	Status
	...	Filter	Filter	Filter	Filter
1	3	Walk Dog	Walk Max around the park	2026-07-16	Not Started
2	4	Reply to mountain of emails	Stop procrastinating	2026-07-25	In Progress
3	5	Send letter	Send off those bills	2026-07-10	Completed
4	6	Milk cows	Milk my cows	2026-07-30	Not Started
5	7	Rob bank	If i add this in disconnected ...	2026-07-20	Not Started
6	8	Finish assignment	Complete the database section	2026-08-01	In Progress
7	9	Submit assignment	Upload the code, report, and ...	2026-08-08	Not Started

Screenshot of the CSV file content

```

exported-tasks.csv
Id,Title,Description,DueDate,Status
3,Walk Dog,Walk Max around the park,2026-07-16,Not Started
4,Reply to mountain of emails,Stop procrastinating,2026-07-25,In Progress
5,Send letter,Send off those bills,2026-07-10,Completed
6,Milk cows,Milk my cows,2026-07-30,Not Started
7,Rob bank,"If i add this in disconnected mode, no one will know!",2026-07-20,Not Started
8,Finish assignment,Complete the database section,2026-08-01,In Progress
9,Submit assignment,"Upload the code, report, and test evidence",2026-08-08,Not Started

```

I confirm that the data from the database was exported correctly to the CSV file.

Please use the check list below to check if you have got everything ready before submitting the assessment for review.

Part 3 - Submission check list

Assessment document with all questions answered

All source codes

Section F – Feedback to Learner

Has the learner successfully completed this assessment task? Yes ☐ No ☐

Assessor feedback (as appropriate): Insert feedback

Resubmission allowed: Yes ☐ No ☐ Resubmission due date: Insert date

Assessor name: Jefi Varghese

Assessor signature:

Learner acknowledgement:



Learner Signature:

Date: 16/07/2026

Generative AI was used to assist across some areas for brainstorming and clarification.

